

Authenticated Blocklisted Systems with Applications to Content Moderation

Anonymous Author(s)

Abstract

An auditable content moderation scheme enables a file server to detect problematic content, e.g., CSAM, in encrypted files uploaded by a client(s). In such a system, the client uploads the encrypted file along with cryptographic metadata that allows the server to decrypt the file if the content matches an item in a blocklist. Otherwise, if the content is not on the blocklist, then the metadata does not reveal any information about the encrypted content. We present new designs for auditable content moderation schemes that (i) enable multiple file servers to use a common, approved blocklist that they cannot undetectably modify; (ii) offer server-client communication and client compute costs that scale sublinearly in the number of elements in the blocklist; and (iii) generalize to various blocklist types and supports exact and fuzzy matching of content. We demonstrate with benchmarks that our schemes are practical and outperform the state of the art.

Keywords

Secure multiparty computation, homomorphic signature, oblivious PRF, encrypted content moderation

1 Introduction

It is estimated that almost 50% of the world’s data will reside on remote services, e.g., public clouds, by the end of 2025 [30]. Unfortunately, storage services are often misused to store harmful content, e.g., child-sexual-abuse-material (CSAM) [45]. In response, large storage providers like Apple, Google, and Amazon have their own mechanisms to identify such harmful content [3, 4, 20]. Typically, these services use blocklists curated by trusted agencies, e.g., the National Center for Missing & Exploited Children (NCMEC).

However, the task is more challenging with end-to-end encryption (E2EE) [33] because it precludes the service provider from implementing simple content sanitization mechanisms that has proved reasonably effective (so far) as a deterrent and an effective method to catch “easy” cases of proliferation [1]. This has raised serious concerns among child safety groups that rely on tips from service providers to detect proliferation of CSAM [1].

To mitigate this tension, in 2021, Apple proposed a mechanism to detect harmful content in end-to-end encrypted photos uploaded to iCloud. The system, called the Apple PSI [8], leveraged Apple’s

proprietary perceptual hashing technology, NeuralHash, to generate a hash of the content. The hash is then compared privately against a blocklist curated by Apple. If there is a match, then the iCloud server learns the decryption key. Sophisticated attackers can still circumvent detection by obfuscating their image, e.g., by “double-encrypting” it before uploading to iCloud, or tamper with the client-side software itself. Nonetheless, the Apple PSI mechanisms provided several important safeguards: i) it allowed the service provider to perform basic content sanitization while supporting E2EE, ii) it could act as a deterrent against bulk uploads by unsophisticated clients [1], and iii) it relieves the service provider of legal liabilities due to publicly hosting CSAM.

Ultimately, the scheme was not deployed due to obvious concerns that it could be misused for oversight and surveillance [14]. Indeed, Apple’s involvement in creating the blocklist, and the lack of mechanisms that allow trusted parties, e.g., NCMEC, to authenticate the blocklist, is problematic [6, 10]. Second, the mechanism was tied exclusively to NeuralHash, which can easily produce false positives [27]; unfortunately, adapting the system to other kinds of perceptual hashes is non-trivial. While the first problem can be addressed through cryptographic mechanisms [1], the latter remains unaddressed to the best of our knowledge.

In light of the discussion above, in this paper we are interested in resolving the following technical question

Can we design a content moderation scheme for end-to-end encrypted data stores with minimal trust assumptions that allows service providers to perform the same level of content sanitization as possible today without end-to-end to encryption?

Authenticated blocklists & distributed trust: We envision a typical setting where a group of independent parties (auditors) with semi-honest majority, e.g., law enforcement, and child safety groups, publish a common (obfuscated) blocklist. Such blocklists are not uncommon, e.g., the NCMEC and FBI have published lists of missing children and persons [19]. We expect that a threshold number of these independent auditors will behave honestly, and posit that having a common blocklist curated by independent agencies, will build more trust in the system than individual blocklists published by self-interested service providers.

Technically, the common blocklist is published as an unforgeable but publicly verifiable *blocklist authentication token*. The token hides the blocklist elements from the clients, so as to ensure that the checks are not trivially circumvented and victim identities are not publicly revealed. During uploads, the *blocklist authentication token* is used by clients to generate content-specific metadata in the form of a *content verification token*, which is provided to the servers (along with the encrypted data) and allows checks against the blocklist. The *content verification token* is hiding in the sense that if the uploaded content is not in the blocklist, then no information is revealed to the server. We realize this design to allow for either *exact matching* of content, where an equality test between content

¹Of course, even if a service does not explicitly support end-to-end encryption, a motivated attacker can still store and distribute CSAM by encrypting the content; it is unclear whether any satisfactory solution can solve this problem without completely violating client-side privacy.



hashes suffices, or *fuzzy matching*, where a distance is computed between content embeddings to estimate similarity [46].

Distributed, randomizable set-homomorphic signatures: We design the *blocklist authentication token* using a new cryptographic primitive, a *randomizable set-homomorphic signature*. A set homomorphic signature [24] enable signing a set of items such that given a signature and the public key, a party can compute a signature on any subset of the items. We further extend the definition with a *randomizing* function. Informally, given a signature on a set $\mathcal{B}$, applying this function to the homomorphically computed signature on $\mathcal{B} \setminus \{y\}$ ensures that the resulting value hides y when $y \notin \mathcal{B}$, even when $\mathcal{B}$ is known. However, when $y \in \mathcal{B}$, the function produces a signature on $\mathcal{B} \setminus \{y\}$ that can be verified efficiently.

We use the auditors' signature on the blocklist $\mathcal{B}$ as the *blocklist authentication token*, thus ensuring unforgeability and public verifiability. Then, the client derives the *content verification token* by homomorphically computing the signature on $\mathcal{B} \setminus \{y\}$, and applying the randomizing function to this signature. If $y \in \mathcal{B}$, the server learns y as it can verify the signature, but otherwise the randomizing function hides y from the server.

We propose a new construction for a randomizable set homomorphic signature scheme under the strong QR-RSA assumption that produces a constant-sized signature, *independent of the blocklist size*. The scheme supports distributed key generation and allows a group of independent auditors with semi-honest majority to sign and update the blocklist in a distributed fashion when the private key is secret-shared among the auditors. We leverage this construction to design a content moderation protocol with optimal client costs. A key feature of our construction is a *fast path* for identifying safe content that eliminates most of the expensive computation (both for the client and the server) required for a full verification, when an (encrypted) uploaded file is not on the blocklist. Thus, our protocol has modest average case costs, since presumably a large portion of the content uploaded is not on the blocklist.

Evaluation: Our exact-matching scheme is highly efficient. The fast path verification takes less than 8ms and a full verification takes roughly 430 ms for a blocklist with million elements. Assuming that realistically less than 10% of the content would require a full check, because uploading CSAM is atypical user behavior, our protocol on average takes 43ms, **12× faster** than the state-of-the-art [26] that operates under a more restrictive threat model with a semi-honest server. Our protocol also has **50×** lower communication costs. While the Apple PSI system does not report performance, we estimate it has similar throughputs but with higher client storage costs and produces false negatives.

Fuzzy matching takes less than 600ms for the fast path check, and 61 seconds for a full check against a blocklist with 1 million items, with false-negative rate of < 0.08 . The fast path is **5×** faster than the state-of-the-art [26] with lower overall communications costs by a factor of **25×**.

Limitations & risks of content moderation: The primary motivation of this work is to develop new cryptographic mechanisms for content moderation, but we also acknowledge the limitations and risks of deploying such a system. First, the efficacy of any content moderation system that only relies on server-side checks (as with our work and all existing designs) is limited to detecting content

that is not obfuscated by the client before uploads e.g., by locally encrypting the data. This excludes a sizable set of attackers. However, evidence shows that even this basic level of monitoring is important for slowing down proliferation of CSAM []. Service providers today are limited to only identifying unencrypted harmful content, as a motivated attacker can still provide an encrypted input even if the service does not support E2EE. The technical solutions in this paper extend these monitoring capabilities while also allowing the service provider to support end-to-end encryption.

Second, as with any content moderation system, there are risks of misuse for surveillance. Technically, our solution addresses this problem by distributing trust among multiple independent agencies, but we acknowledge that this may not suffice against a powerful adversary, e.g., a nation state who may corrupt all the auditors. Thus, further discussions and potential mitigations are necessary before any content moderation system, including ours, is widely deployed. These discussions are out of scope for this paper.

2 Related Work

In this work, we are interested in content moderation schemes for encrypted file stores that use blocklists. Content moderation has been studied generally for network communication and encrypted traffic streams, but these solutions cannot be immediately applied to file stores without incurring high costs or leaking matching patterns [31, 32, 42, 44]. We refer the interested reader to an excellent survey [39] on content moderation for further details

Table 1 compares the state of the art schemes with our work. We exclude preprocessing and blocklist creation costs for all the schemes since they are linear in the size of the blocklist. Instead we focus on the online costs during file uploads. Auditability refers to whether the scheme allows a set of independent auditors to create and verify the blocklist. Our protocols support a *fast path* for verification assuming that most of the uploaded content will not be blocked; we present the average case and worst case costs.

Kulshrestha and Mayer [26] introduced a system with a blocklist of perceptual hashes of CSAM content. The protocol leverages a single server PIR scheme, and supports both exact and approximate matching against the blocklist. *The protocol only supports a semi-honest service provider, and cannot be trivially extended to provide malicious security without incurring large client costs, e.g., by using an authenticated PIR [15]*. The scheme also does not allow independent auditors to verify that the blocklist has been created honestly. In contrast, we do not assume any trust on the service provider. Finally, the approximate match scheme only works for a specific type of content embedding with empirically determined parameters and high false negatives ($> 17\%$). The scheme cannot be trivially extended to other types of embeddings.

Apple developed a mechanism to detect CSAM content in encrypted files uploaded to iCloud [4] using the NeuralHash model [8]. A core criticism of this work is that the blocklist is created solely by Apple. This criticism was addressed by Scheffler et al. [38] by leveraging an aggregated signature scheme. *Nonetheless, the mechanism does not generalize to other types of content hashes, including ones that require fuzzy matching*. This is a drawback because NeuralHash without fuzzy matching is easy to circumvent, and also suffers from false positives [27, 43].

Table 1: Encrypted content moderation systems for CSAM detection. n : blocklist size, $\epsilon \in (0, 1/2)$, ℓ : vector length, SH: semi-honest, M: malicious, FP: false positive rate, FN: false negative rate, Avg: average case cos, WC: worst case cost

Type of Matching	System Name	Audibility	Threat model		Asymptotic Costs				Accuracy	
			Server	Client	Client Comp.	Comm. Volume	Client Storage	Server Comp.	FP	FN
Exact	Private Hash Matching [26]	N/A	SH	SH	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	0	0
	Apple PSI [8] + Agg. Signature [38]	✓	M	SH	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	0	> 0
	This work	✓	M	SH	Avg: $\mathcal{O}(1)$, WC: $\mathcal{O}(\log n)$	$\mathcal{O}(n^\epsilon)$	$\mathcal{O}(n^\epsilon)$	Avg: $\mathcal{O}(1)$, WC: $\mathcal{O}(n)$	0	0
Fuzzy	Private Hash Matching [26]	N/A	SH	SH	$\mathcal{O}(\sqrt{n\ell})$	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(1)$	$\mathcal{O}(n\ell)$	0	> 0
	This work	✓	M	SH	$\mathcal{O}(n^\epsilon \ell)$	$\mathcal{O}(n^\epsilon \ell)$	$\mathcal{O}(n^{1+\epsilon})$	Avg: $\mathcal{O}(n^\epsilon)$, WC: $\mathcal{O}(n^\epsilon \ell)$	0	> 0

Gap in research. We observe from Table 1 that there is no scheme that provides auditability, supports both exact and fuzzy matching, and protects against a malicious server, while also ensuring that the client compute and communication costs remain sublinear in the blocklist size. Our work addresses these requirements.

3 Model

3.1 Problem

A content moderation scheme enables a file server to detect harmful content, e.g., CSAM, in encrypted files uploaded by a client(s). The client provides metadata that allows the server to decrypt the file if the content matches an item in a *blocklist*. Otherwise, if the content is not on the blocklist, then the metadata does not reveal any information about the encrypted content. Typically, a blocklist is created from a content hash [8] or an *embedding* [17, 28, 37]. In this work, we envision that the blocklist is jointly created by a group of *auditors*. Clients can verify the authenticity of the blocklist using some public cryptographic token, e.g., a signature. Further, clients do not learn the items on the blocklist, as otherwise they can try simple transformations to evade detection.

Depending on the type of items in the blocklist, content matching may either be in the form of *exact matching* or *fuzzy matching*. Exact matching is an equality test between an input and the items in the blocklist, while fuzzy matching typically computes a distance between content embeddings and compares against a threshold. In both cases, the goal is to detect content that is similar (and not just identical) because simple transformations like adding a filter will make the content different enough from a nearby blocklist element. Typically, exact matching is performed over learned content hashes, e.g., a NeuralHash [4] that derives strings or integers from the content such that two similar items hash to the same value with high probability. On the other hand, fuzzy matching leverages an embedding function that maps content x_1 and x_2 to corresponding elements in a metric space $\mathcal{M}$ with distance d . If the content is similar then $\|embed(x_1) - embed(x_2)\|_d \leq t$ with high probability, where t is a threshold. Both mechanisms may introduce false positives and false negatives that can be tuned using parameters.

3.2 Threat Model

Our content moderation system has three parties.

- (1) **Auditors:** There is a group of agencies that publish and/or verify the items on the blocklist. The auditors may include representatives from law enforcement, social rights groups, independent journalists, etc. Some of the auditors may also be

untrusted, i.e., they may collude with the file servers. For this work, we assume that a majority of the auditors are honest².

- (2) **File server:** The server has the blocklist, although it may not necessarily participate in creating it. The file server may behave maliciously and collude with the dishonest auditors to learn information in the encrypted files. However, it ensures integrity and availability of the files.
- (3) **Client:** The client is semi-honest, i.e., it will follow protocols. While at first glance, it might appear that the assumption is too strong, this model has largely proven successful in detecting content like CSAM, and acting as a deterrent against less sophisticated users [8, 26, 38]. Moreover, sophisticated users capable of circumventing checks, e.g., by modifying the client-side code, can also use unmonitored channels for storing harmful content [8, 26, 38].

On the semi-honest client assumption: Before proceeding further with our security goals, we pause to consider the practicality of our security model. Of course, the obvious objection is that clients who upload CSAM may not follow protocols since circumventing detection is ultimately their goal. Indeed, a malicious client can leverage tools for adversarial noising [41], or “double-encrypt” their files so that the client-side software only observes random data. Unfortunately, it is unclear how such attacks can be prevented without using more sophisticated cryptographic tools such as general purpose verifiable computation [18], or trusted execution environments [23]. Both approaches add overheads/assumptions that are not be suitable for resource-constrained client devices. Consequently, **all encrypted content moderation tools for detecting CSAM, including Apple’s (proposed but now defunct) solution to detect CSAM on iCloud assume a semi-honest client.**

Moreover, there are several reasons to consider semi-honest clients a realistic threat. First, there is ample evidence that CSAM is often uploaded by unsophisticated users “in the clear” [34, 35]. However, if cloud providers support end-to-end encryption by default (as it is emerging as a de-facto standard), the opportunities for detecting even such obvious cases are hindered. A tool for a semi-honest client will enable servers to at least perform a baseline level of detection. Second, our mechanisms are compatible with content embeddings that are resilient to adversarial image manipulations [46] by the client while *also tolerating a malicious server*. Existing works do not simultaneously consider both of these potential threats (Table 1), and therefore our work is an important first step towards designing schemes for a realistic threat model.

²The honest majority assumption is an artifact of the cryptographic tool we use in our scheme but it is not a fundamental requirement of the general framework.

$\mathcal{F}_{\text{ExactMod}}$: Auditable Content Moderation with Exact Matching Parameters: η auditors $A_1, \dots, A_\eta$, a server SERVER and a client CLIENT. A signature scheme $\Pi_{\text{Sig}} = (\text{SGEN}, \text{SSIGN}, \text{SVRFY})$. False negative rate $\epsilon_{\text{fnr}} \in (0, 1)$ and false positive rate $\epsilon_{\text{fpr}} \in (0, 1)$. Creating blocklist: (1) Upon receiving the message $(\text{CreateBlocklist}, A_k, \mathcal{B}, sk_k)$ from A_k, store the blocklist and the auditor's private key (2) After receiving the CreateBlocklist message from all auditors, abort if the blocklists submitted by the auditors are not the same. Otherwise, output an authenticated and confidential blocklist $\mathcal{T}_{\mathcal{B}}$ created from $\mathcal{B}$, and signatures under the auditor's keys $\{\text{SSIGN}_{sk_k}(\mathcal{T}_{\mathcal{B}})\}_{k \in [1, \eta]}$. Store $\mathcal{B}$ Verifying uploads: (3) Initialize a list RetList (4) Upon receiving $(\text{UPLOADFILE}, \text{ID}, C_{\text{DATA}})$ from CLIENT, store $(\text{ID}, C_{\text{DATA}})$ in RetList (5) Upon receiving $(\text{CHECKFILE}, \text{ID}, C'_{\text{DATA}})$ from SERVER, (a) If there is no entry $(\text{ID}, _)$ in KeyList, then abort. (b) If there is an entry $(\text{ID}, C'_{\text{DATA}}) \in \text{RetList}$, and $C'_{\text{DATA}} \in \mathcal{B}$, then return 1 with probability at least $1 - \epsilon_{\text{fnr}}$. (c) If there is an entry $(\text{ID}, C'_{\text{DATA}}) \in \text{RetList}$, and $C'_{\text{DATA}} \in \mathcal{B}$, then return 0 with probability at least $1 - \epsilon_{\text{fnr}}$.	$\mathcal{F}_{\text{FuzzyMod}}$: Auditable Content Moderation with Fuzzy Matching Parameters: η auditors $A_1, \dots, A_\eta$, a server SERVER and a client CLIENT. A signature algorithm $\Pi_{\text{Sig}} = (\text{SGEN}, \text{SSIGN}, \text{SVRFY})$. Threshold t for a metric distance $\ \cdot\ _d$. A false positive rate $\epsilon_{\text{fpr}} \in (0, 1)$ and false negative rate $\epsilon_{\text{fnr}} \in (0, 1)$. Creating blocklist: (1) Upon receiving the message $(\text{CreateBlocklist}, A_k, \mathcal{B}, sk_k)$ from A_k, store the blocklist and the auditor's private key (2) After receiving the CreateBlocklist message from all auditors, abort if the blocklists submitted by the auditors are not the same. Otherwise, output an authenticated and confidential blocklist $\mathcal{T}_{\mathcal{B}}$ created from $\mathcal{B}$, and signatures under the auditor's keys $\{\text{SSIGN}_{sk_k}(\mathcal{T}_{\mathcal{B}})\}_{k \in [1, \eta]}$. Store $\mathcal{B}$ Verifying uploads: (3) Initialize a list RetList (4) Upon receiving $(\text{UPLOADFILE}, \text{ID}, C_{\text{DATA}})$ from CLIENT, store $(\text{ID}, C_{\text{DATA}})$ in RetList (5) Upon receiving $(\text{CHECKFILEFUZZY}, \text{ID}, C'_{\text{DATA}})$ from SERVER, (a) If there is no entry $(\text{ID}, _)$ in RetList, then abort. (b) If there is an entry $(\text{ID}, C'_{\text{DATA}}) \in \text{RetList}$ • Check if there is some entry $b \in \mathcal{B}$ such that $\ b - C'_{\text{DATA}}\ _d \leq t$. If so, return 1 with probability $\geq 1 - \epsilon_{\text{fnr}}$. Otherwise return 0. • If for all $b \in \mathcal{B}$, $\ b - C'_{\text{DATA}}\ _d > t$, return 0 with probability $\geq 1 - \epsilon_{\text{fpr}}$. Otherwise return 1.
--	---

Figure 1: Ideal functionality for auditable content moderation with exact and fuzzy matching

3.3 Security Goals

The core building block of our content moderation schemes are blocklists that can be stored on the client while ensuring: i) **confidentiality**: clients do not learn the constituent blocked elements, and ii) **unforgeability**: only the auditors can create/modify the blocklist. We call such blocklist designs *authenticated and confidential blocklists* (ACBs). The ideal functionality for auditable content moderation using such blocklists is presented in Fig. 1. $\mathcal{F}_{\text{ExactMod}}$ supports exact matching against the blocklist, while $\mathcal{F}_{\text{FuzzyMod}}$ supports a metric distance based fuzzy match.

The auditors invoke the functionality with the CreateBlocklist to create an ACB, $\mathcal{T}_{\mathcal{B}}$, from a given set of blocked items, $\mathcal{B}$, and a signature under the auditors' private keys. The ACB and the signatures are published and stored by the client. During an upload, the client invokes the functionality with the UPLOADFILE message providing an input C_{DATA} which is derived from the file content it wants to upload. The functionality stores C_{DATA} along with a file identifies C_{DATA} can be either a hash of the file, or a metric space embedding depending on the type of matching. After the client uploads the file, the server checks whether $C_{\text{DATA}} \in \mathcal{B}$ by invoking the functionality with the CHECKFILE (resp. CHECKFILEFUZZY) message. The functionality returns the result of the check with false positive rate ϵ_{fpr} and false negative rate ϵ_{fnr} .

Remark 3.1. The need for blocklist confidentiality is motivated by real-world requirements where child safety groups and hosting platforms have opposed releasing public blocklists since they can be considered unlawful and reveal information about victims [9, 26].

4 Preliminaries

Notation: We use $\leftarrow$ to denote assignment of the value on its right to the variable on its left, and $\leftarrow \$$ to sample uniformly from the set on its right and assign the sampled value to the variable on its left. Let $g : \mathcal{K} \times \mathcal{D} \rightarrow \mathcal{R}$ denote a function family with *keyspace* $\mathcal{K}$, *domain* $\mathcal{D}$, and *range* $\mathcal{R}$. For $k \in \mathcal{K}$, we denote by $g_k : \mathcal{D} \rightarrow \mathcal{R}$ the

function with $g_k(d) = g(k, d)$ for each $d \in \mathcal{D}$. We use λ to denote a security parameter.

Definition 4.1 (Strong QR-RSA problem). Assume that $N = pq$ where p and q are random, safe prime numbers, and QR_N is the quadratic residue subgroup of $\mathbb{Z}_N^*$. Then, for $y \leftarrow QR_N$,

$$\begin{aligned} \text{Adv}_N^{\text{QR-RSA}}(\mathcal{A}_{\text{QR-RSA}}) &= \\ \mathbb{P} \left(x \in QR_N \wedge e \geq 2 \wedge \begin{matrix} x^e \equiv y \pmod{N} \end{matrix} \mid (x, e) \leftarrow \mathcal{A}_{\text{QR-RSA}}(N, y) \right) \\ \text{Adv}_N^{\text{QR-RSA}}(t) &= \max_{\mathcal{A}_{\text{QR-RSA}}} \text{Adv}_N^{\text{QR-RSA}}(\mathcal{A}_{\text{QR-RSA}}) \end{aligned}$$

where the probability is over the choice of N and y and random choices by $\mathcal{A}_{\text{QR-RSA}}$, and the maximum is taken over all adversary algorithms $\mathcal{A}_{\text{QR-RSA}}$ that run in total time t .

We will make black-box use of the following primitives

Symmetric key encryption scheme defined by the triple of algorithms $\Pi_{\text{enc}} = (\text{KEYGEN}, \text{ENC}, \text{DEC})$ with key space K . For security, the encryption scheme must satisfy *weak* random key robustness [8]. This property implies that a ciphertext encrypted under a randomly chosen key, cannot be decrypted under a different, randomly chosen key. We formally define the property in App. D.

Additively homomorphic encryption scheme defined by the triple $\Pi_{\text{ahe}} = (\text{AH_KEYGEN}, \text{AH_ENC}, \text{AH_DEC})$. The plaintext space is a group G of prime order. We require that the encryption scheme is homomorphic under the group operation. That is, there is an efficient algorithm to compute a ciphertext $\text{Enc}_{pk_e}(m_1 + m_2)$ from pk , $c_1 = \text{Enc}_{pk_e}(m_1)$, and $c_2 = \text{Enc}_{pk_e}(m_2)$; we denote this operation by $c_1 \vdash_{pk} c_2$. The security assumption for encryption to which we reduce is IND-CPA security [7], formally defined in App. D.1

Digital signature scheme defined by the triple of algorithms $\Pi_{\text{sig}} = (\text{SGEN}, \text{SSIGN}, \text{SVRFY})$ that satisfies unforgeability under chosen message attack, formally defined in Fig. 3 in the Appendix.

5 Blocklist Constructions

In this section, we will discuss constructions for authenticated and confidential blocklists (ACB) leveraging new cryptographic building blocks. To securely realize the CHECKFILE and CHECKFILEFUZZY interfaces in the ideal functionality, we will ensure that an ACB supports **oblivious matching**: the client can obviously match its input against items in the blocklist and reveal the result to the server; If the client's input is not in the blocklist, then the result of the match does not reveal any information.

We formally define the interfaces exposed by an ACB

- **Publish**: This is an interactive mechanism between the auditors to create a blocklist from a common set of items $\mathcal{B} \leftarrow \{b_1, \dots, b_n\} \subseteq \mathcal{U}$ and publish the output $\mathcal{B}^{(\text{blind})}$. As discussed before, an ACB ensures
 - *Confidentiality*: for any PPT adversary $\mathcal{A}$

$$\mathbb{P}(\mathcal{A}(\mathcal{B}^{(\text{blind})}) = b \wedge b \in \mathcal{B}) \leq 1/|\mathcal{U}|$$

- *Unforgeability*: The auditors sign $\mathcal{B}^{(\text{blind})}$ (and other relevant information) under their own keys.

- **Preprocess**: This is an optional mechanism between the client and the server where they can interactively process the published blocklist to create a representation $\mathcal{T}_{\mathcal{B}}$ that support oblivious matching. $\mathcal{T}_{\mathcal{B}}$ is stored on the client side for future searches. The preprocessing may require secrets from both the client and the server. We also support constructions where preprocessing is not necessary to support matching. In that case, $\mathcal{T}_{\mathcal{B}} = \mathcal{B}^{(\text{blind})}$.
- **Search**: This is a mechanism between the client and the server where given an input y from the client, the functionality outputs either y or $\perp$ to the server depending on whether $y \in \mathcal{B}$, and a predefined false positive rate and a false negative rate.

$$\mathbb{P}(\text{Search}(y, \mathcal{T}_{\mathcal{B}}) = y \mid y \in \mathcal{B}) \geq 1 - \epsilon_{\text{fnr}} \quad (1)$$

$$\mathbb{P}(\text{Search}(y, \mathcal{T}_{\mathcal{B}}) = \perp \mid y \notin \mathcal{B}) \leq 1 - \epsilon_{\text{fpr}} \quad (2)$$

, where $\epsilon_{\text{fnr}} \in (0, 1)$ is the false negative rate, and $\epsilon_{\text{fpr}} \in (0, 1)$ is the false positive rate. Crucially, if the client's input is not on the blocklist, then no additional information is revealed about y to the server. The mechanism also returns a $\{0, 1\}$ to the client to indicate whether there was a match.

- **TSearch**: This is an optional mechanism between client and server where given a set of inputs $Y \leftarrow \{y_1, \dots, y_\ell\}$ from the client, and a public threshold t , the functionality outputs a value in $\{0, 1\}$

$$\mathbb{P}(\text{TSearch}(Y, \mathcal{T}_{\mathcal{B}}) = 1 \mid |Y \cap \mathcal{B}| \geq t) \geq 1 - \epsilon_{\text{fnr}}$$

$$\mathbb{P}(\text{TSearch}(Y, \mathcal{T}_{\mathcal{B}}) = 0 \mid |Y \cap \mathcal{B}| < t) \geq 1 - \epsilon_{\text{fpr}}$$

, where $\epsilon_{\text{fnr}} \in (0, 1)$ is the false negative rate, and $\epsilon_{\text{fpr}} \in (0, 1)$ is the false positive rate. Crucially, if the client's input is not on the blocklist, then no additional information is revealed about y to the server. The mechanism also returns a $\{0, 1\}$ to the client to indicate whether there was a match.

$\mathcal{F}_{\text{authOPRF}}$: Ideal Functionality for Authenticated OPRF

Parameters: η auditors $A_1, \dots, A_\eta$, a sender CLIENT and a receiver SERVER. A pseudorandom function $\text{PRF} : \mathcal{K}_{\text{PRF}} \times \mathcal{U} \rightarrow \{0, 1\}^\lambda$, a signature scheme $\Pi_{\text{Sig}} = (\text{SGEN}, \text{SSIGN}, \text{SVRFY})$, a hash function $h : \{0, 1\}^* \rightarrow G$

Inputs: The auditors have a common list of item $\mathcal{B} \subseteq \mathcal{U}$; auditor has public-private key pair $(pk_k, sk_k)_{k \in [\eta]}$ for Π_{Sig} corresponding to a public key pk . CLIENT has the key κ_{PRF} for PRF.

Functionality:

- Upon receiving the message $(\text{Publish}, A_k, \mathcal{B}, sk_k)$ from A_k , store the list of item $\mathcal{B}$ and the auditor's private key sk_k
- After receiving the Publish message from all auditors, abort if the list of items $\mathcal{B}$ submitted by the auditors are not the same. Otherwise,
 - (1) Initialize a key-value store KV.
 - (2) For each $b \in \mathcal{B}$, add an entry $\langle b, r \rangle$ to KV, where b is the key and $r \leftarrow \{0, 1\}^\lambda$
 - (3) Add each value r in KV to a set $\mathcal{B}'$.
 - (4) Sign a hash of $\mathcal{B}'$ under the private keys $sk_k \leftarrow \text{SSIGN}_{sk_k}(h(\mathcal{B}'))$
 - (5) Output $\mathcal{B}'$ and $\{\Sigma_k\}_{k \in [1, \eta]}$
- Upon receiving the message $(\text{Eval}, \mathcal{B}', \kappa_{\text{PRF}})$ from CLIENT, for each $\langle b, r \rangle \in \text{KV}$, output to SERVER $\langle b, \text{PRF}_{\kappa_{\text{PRF}}}(r) \rangle$.

Figure 2: $\mathcal{F}_{\text{authOPRF}}$: Authenticated OPRF

Content moderation from authenticated blocklists: A secure ACB construction almost immediately provides a secure realization of $\mathcal{F}_{\text{ExactMod}}$. We outline the steps below

- (1) The auditors invoke Publish with the blocklisted items $\mathcal{B}$, which outputs $\mathcal{B}^{(\text{blind})}$ and the signatures under the auditors' private keys. The client downloads $\mathcal{B}^{(\text{blind})}$ and verifies the signatures.
- (2) If the ACB supports preprocessing, then the client and server invoke Preprocess to produce $\mathcal{T}_{\mathcal{B}}$ that is stored on the client. If there is no preprocessing, then $\mathcal{T}_{\mathcal{B}} = \mathcal{B}^{(\text{blind})}$.
- (3) When a client wants to upload content, the client and server invoke Search which enables the server to determine if the client's content is on the blocklist.

We will introduce three different types of authenticated blocklists, and use these constructions later in Sec. 5 to design our content moderation schemes.

5.1 Bloom Filter Based Blocklist

A Bloom filter provides a succinct representation of a set with storage costs that are sublinear in the set size. A Bloom filter is ideally suited for a large blocklist since it is stored on the client. However, a Bloom filter does not provide confidentiality of the blocklisted items, as required by our functionality³.

Our key idea to introduce confidentiality is to construct the Bloom filter with items that are obfuscated against the client but still enables efficient searches. This is supported

5.1.1 Authenticated OPRF. We define a functionality called *authenticated OPRF*, that extends the definition of an OPRF to a three party functionality $\mathcal{F}_{\text{authOPRF}}$ (Fig. 2), where a group of η auditors with semi-honest majority jointly sign a list of inputs to a pseudorandom function. $\mathcal{F}_{\text{authOPRF}}$ receives the key for the PRF from the client, who acts as the sender. The receiver of the PRF evaluations is the server, who does not input anything.

³We note that a Bloom filter without confidentiality was considered in prior work but assumed to be unsatisfactory.

Realizing $\mathcal{F}_{\text{authOPRF}}$: We realize $\mathcal{F}_{\text{authOPRF}}$ by leveraging the the 2HashDH PRF construction [22] in a group where the OM gap-DH problem is hard. The construction is $\text{PRF}_{\kappa_{\text{PRF}}}^{\text{C}}(x) = H_2(x, H_1(x)^{\kappa_{\text{PRF}}^{\text{C}}})$, where H_1, H_2 are modeled as random oracles. The full protocol is in Fig. 8 in App. C. In brief,

- (1) The auditors “blind” the blocklist elements $\mathcal{B}^{(\text{blind})} \leftarrow \{H_1(b_i)^{r_i}\}_{b_i \in \mathcal{B}}$ and the server is provided the blinding factors $r_1, \dots, r_n$.
- (2) The auditors publish $\mathcal{B}^{(\text{blind})}$ and sign (a hash of) $\mathcal{B}^{(\text{blind})}$ under their private keys. The client obtains $\mathcal{B}^{(\text{blind})}$ and the auditors’ signatures.
- (3) After verifying the signatures, the client exponentiates the blinded elements $\{(H_1(b_i)^{r_i})^{\kappa_{\text{PRF}}^{\text{C}}}\}_{b_i \in \mathcal{B}}$ and sends them to the server.
- (4) The server removes the blinding and computes $\{H_2(b_i, H_1(b_i)^{\kappa_{\text{PRF}}^{\text{C}}})\}_{b_i \in \mathcal{B}}$.

5.1.2 Blocklist construction. We can create an authenticated Bloom-filter-based blocklist using $\mathcal{F}_{\text{authOPRF}}$ as follows

Algorithm 1 Bloom Filter Based Blocklist

```

1: procedure Publish( $\mathcal{B}$ )
2:   Auditor  $A_k$  samples a key pair for a standard digital signature scheme
    $\langle pk_k, sk_k \rangle$ 
3:   Given a blocklist  $\mathcal{B}$ , the auditors invoke  $\mathcal{F}_{\text{authOPRF}}$  with the Publish message.
   Auditors sign the output  $\mathcal{B}^{(\text{blind})}$   $\{\text{SSIGN}_{sk_k}(h(\mathcal{B}^{(\text{blind})}))\}_{k \in [1, \eta]}$ 
4:   Publish  $\mathcal{B}^{(\text{blind})}$ ,  $\{\text{SSIGN}_{sk_k}(h(\mathcal{B}^{(\text{blind})}))\}_{k \in [1, \eta]}$ 

5: procedure Preprocess( $\mathcal{B}^{(\text{blind})}$ )
6:   CLIENT downloads  $\mathcal{B}^{(\text{blind})}$ ,  $\{\text{SSIGN}_{sk_k}(h(\mathcal{B}^{(\text{blind})}))\}_{k \in [1, \eta]}$  and verifies
   the signature. Then, it invokes  $\mathcal{F}_{\text{authOPRF}}$  with Eval with  $\mathcal{B}^{(\text{blind})}$  and  $\kappa_{\text{PRF}}^{\text{C}}$ .
7:    $\triangleright$  Upon receiving  $\{\langle b, \text{PRF}_{\kappa_{\text{PRF}}^{\text{C}}}(b) \rangle\}_{b \in \mathcal{B}}$  from  $\mathcal{F}_{\text{authOPRF}}$ , SERVER implements  $\Leftarrow$ 
8:   Initialize  $\zeta = n/\log n$  bins  $\text{Bin}_1, \dots, \text{Bin}_{\zeta}$ 
9:   for each  $\langle b, \text{PRF}_{\kappa_{\text{PRF}}^{\text{C}}}(b) \rangle$  do
10:    Add  $\text{Bin}_k \leftarrow \text{Bin}_k \cup \langle b, \text{PRF}_{\kappa_{\text{PRF}}^{\text{C}}}(b) \rangle_{b \in \mathcal{B}}$  where  $\text{PRF}_{\kappa_{\text{PRF}}^{\text{C}}}(b) \equiv k \pmod{\zeta}$ 
11:   for each bin  $\text{Bin}_k \in \{\langle b, \text{PRF}_{\kappa_{\text{PRF}}^{\text{C}}}(b) \rangle\}_{b \in \mathcal{B}}$  do
12:     Initialize a Bloom filter  $bf_k$  with false positive rate  $\epsilon_{bf}$ 
13:     for each  $\langle b, \text{PRF}_{\kappa_{\text{PRF}}^{\text{C}}}(b) \rangle \in \text{Bin}_k$  do
14:        $bf_k \leftarrow bf_k \cup b$ 
15:        $bf_k \leftarrow bf_k \cup \text{PRP}_{\kappa_{\text{PRP}}^{\text{S}}}(\text{PRF}_{\kappa_{\text{PRF}}^{\text{C}}}(b))$ 
16:   Sent  $\mathcal{T}_{\text{BF}} \leftarrow \{\text{ENC}_{\kappa}(bf_1), \dots, \text{ENC}_{\kappa}(bf_{\zeta})\}$  to CLIENT

17: procedure Search( $y, \mathcal{T}_{\text{BF}}$ )
18:   if  $y \notin \mathcal{L}$  then
19:     CLIENT sets  $z \leftarrow \text{PRF}_{\kappa_{\text{PRF}}^{\text{C}}}(y)$ 
20:   else
21:     CLIENT sets  $z \leftarrow \{0, 1\}^{\lambda}$ 
22:   CLIENT computes  $k \leftarrow z \pmod{\zeta}$  and sends to SERVER  $z$  and  $\text{ENC}_{\kappa}(bf_k)$ .
23:   SERVER decrypts  $\text{ENC}_{\kappa}(bf_k)$  and checks whether  $\text{PRP}_{\kappa_{\text{PRP}}^{\text{S}}}(z) \in bf_k$ . If so, it
   return 1 to CLIENT, and otherwise returns 0.
24:   CLIENT adds  $\mathcal{L} \leftarrow \mathcal{L} \cup \{y\}$  if SERVER outputs 1

```

Costs: The costs of creating and searching are as follows

- CreateBlocklist has $\mathcal{O}(n)$ cost for the auditors, client and server. This is a one-time cost
- Search has $\mathcal{O}(1)$ cost for the client and the server
- The total storage cost for the client is $\sum_{k \in [1, \zeta]} |bf_k|$

Security: The security of the ACB is due to the following

- Confidentiality and unforgeability is ensured by the security of $\mathcal{F}_{\text{authOPRF}}$, and the IND-CPA security of the encryption scheme.

5.2 Homomorphic Signature Based Blocklist

There are two drawbacks of the Bloom filter based ACB construction: i) the storage overhead for $\mathcal{T}_{\mathcal{B}}$ is $\mathcal{O}(n)$, and ii) the search can produce false positives. In this section, we will design a ACB that has zero false positives and false negatives and constant storage overhead, but with higher computation costs during search. The construction leverages a new cryptographic primitive called a *randomizable set homomorphic signature*. Intuitively, the ACB is constructed from a *set-homomorphic* signature on the accumulated set of blocklisted items [24] under the auditors’ private keys. The signature scheme has homomorphic properties that allows us to compute signatures of subsets of a signed set, as well as the union of two signed sets. We leverage these homomorphic properties to directly use the signature as the ACB and compute on the signature to implement an oblivious match. Importantly, the ACB in this construction is constant-sized making it highly storage efficient.

5.2.1 Randomizable Set-Homomorphic Signatures.

Definition 5.1. A *randomizable set homomorphic signature* over sets drawn from a universe $\mathcal{U}$ is a collection of five polynomial-time algorithms $\Pi_{\text{RH-Sig}} = \langle \text{GEN}, \text{SIGN}, \text{VERFY}, \text{RAND}, \text{AUTH} \rangle$. The key space is $\mathcal{K}$ and the output space is $\mathcal{R}$.

- $\langle pk, sk \rangle \leftarrow \text{GEN}(1^{\lambda})$: Generates a public-private key pair
- $\sigma_S \leftarrow \text{SIGN}_{sk}(S)$: On input a set $S \subseteq \mathcal{U}$, and private key sk generated by GEN, the algorithm produces a signature $\sigma \in \mathcal{R}$.
- $\{0, 1\} \leftarrow \text{VERFY}_{pk}(S, \sigma)$: On input a set $S \subseteq \mathcal{U}$, a signature σ , and the public key pk corresponding to sk , the algorithm outputs 1 if $\sigma = \text{SIGN}_{sk}(S)$. Otherwise, it outputs 0.
- $\langle v', w' \rangle \leftarrow \text{RAND}_{pk}(v)$: On input an arbitrary element from the output space $v \in \mathcal{R}$, the algorithm outputs a *blinded signature* $v' \leftarrow f_{pk}(v, r)$,
 - $f_{pk} : \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R}$ is injective for a fixed r , and the output of f is uniformly distributed for a uniformly sampled r .
 - $H : \mathcal{R} \rightarrow \{0, 1\}^{\lambda}$ is a random oracle
 - $\sigma_{\emptyset} = \text{SIGN}_{sk}(\emptyset)$ is the signature on the null set
 - $r \leftarrow \mathcal{R}$ is uniformly sampled from a space of random coins (defined later).
- $\{v, \perp\} \leftarrow \text{AUTH}_{pk}(S, \langle v', w' \rangle)$: Upon being provided $v' \in \mathcal{R}$ and $w' \in \{0, 1\}^{\lambda}$, the algorithm returns v if $\langle v', w' \rangle$ is an output of $\text{RAND}_{pk}(S)$ for some $v \in S$. Otherwise, it returns $\perp$.

Set Homomorphism: There is an operator $\odot_{pk} : \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R}$ such that given two sets S_1 and S_2 , signatures σ_1 and σ_2 , where $\text{VERFY}_{pk}(S_1, \sigma_1) = 1$ and $\text{VERFY}_{pk}(S_2, \sigma_2) = 1$, it is the case that

$$\text{VERFY}_{pk}(S_1 \cup S_2, \sigma_1 \odot_{pk} \sigma_2) = 1. \quad (3)$$

There is also an operator $\ominus_{pk} : \mathcal{R} \times \Phi(\mathcal{U}) \rightarrow \mathcal{R}$, where $\Phi(\mathcal{U})$ denotes the powerset of $\mathcal{U}$, such that given σ where $\text{VERFY}_{pk}(S, \sigma) = 1$ and given $S' \subseteq \mathcal{U}$,

$$\text{VERFY}_{pk}(S \setminus S', \sigma \ominus_{pk} S') = 1 \quad (4)$$

The function f is homomorphic over the signature space such that for all $S' \subseteq \mathcal{U}$ and $\sigma \in \mathcal{R}$

$$f_{pk}(\sigma, r) \ominus_{pk} S' = f_{pk}(\sigma \ominus_{pk} S', r) \quad (5)$$

Note that (5) holds even when $S' \not\subseteq S$.

Remark 5.1. In our discussions we will denote a signature on the set $S \subseteq \mathcal{U}$ as σ_S for notational simplicity. For any $y \in \mathcal{U}$, the computation $\sigma_{\mathcal{B}} \ominus_{pk} \{y\}$ returns a valid signature on the set $\mathcal{B} \setminus \{y\}$ when $y \in \mathcal{B}$ due to Eq. (4). We denote this signature with $\sigma_{\mathcal{B} \setminus \{y\}}$. However, when $y \notin \mathcal{B}$, $\sigma_{\mathcal{B}} \ominus_{pk} \{y\}$ is an element in the output space $\mathcal{R}$ that is not necessarily equal to $\sigma_{\mathcal{B}}$. As will be evident later, $\sigma_{\mathcal{B}} \ominus_{pk} y$ is a signature on an unpredictable element in $\mathcal{R}$, and with abuse of notation we also denote this element as $\sigma_{\mathcal{B} \setminus \{y\}}$.

Security: A randomizable set homomorphic signature scheme must satisfy two security properties

- **Unforgeability:** The signature scheme must satisfy unforgeability under chosen message attacks (UF-CMA), defined by the experiment $\text{EXPT}_{\Pi_{\text{Sig}}}^{\text{uf-cma}}(\mathcal{A}_{\text{ufcma}})$ in Fig. 3.
- **Indistinguishable signatures under randomization:** For all $y \notin \mathcal{B}$, we want the output of $\text{RAND}_{pk}(\sigma_{\mathcal{B}} \ominus_{pk} \{y\})$ to hide y . We formalize this property as *indistinguishable signatures under randomizations (IND-SR)*, using a CPA-style definition such that for $y_0, y_1 \notin \mathcal{B}$, the outputs of $\text{RAND}_{pk}(\sigma_{\mathcal{B}} \ominus_{pk} \{y_0\})$ and $\text{RAND}_{pk}(\sigma_{\mathcal{B}} \ominus_{pk} \{y_1\})$ are indistinguishable. This is defined by the experiment $\text{EXPT}_{\Pi_{\text{Sig}}}^{\text{ind-sr}[b]}$ in Fig. 3.

5.2.2 RSA-Based Construction. Fig. 5 describes a randomizable set homomorphic signature scheme that is secure under the strong QR-RSA assumption.

Correctness: The correctness of GEN, SIGN and VRFY is obvious. We describe how the signature scheme satisfies the homomorphic properties. All calculation are performed modulo N ; we omit this factor from the description for notational simplicity.

Let S_1, S_2 be two sets and $U_1 \leftarrow \prod_{y \in S_1} H_P(y)$, and $U_2 \leftarrow \prod_{y' \in S_2} H_P(y')$.

Also, let the signatures on S_1 and S_2 be $\sigma_{S_1} \leftarrow g^{1/U_1}$ and $\sigma_{S_2} \leftarrow g^{1/U_2}$ respectively.

- **Union of sets:** We define the homomorphic operator $\odot$ below

$$\begin{aligned} \sigma_{S_1} \odot_{pk} \sigma_{S_2} &= \sigma_{S_1 \cup S_2} = \sigma_{S_1}^b \sigma_{S_2}^a = g^{b/U_1} g^{a/U_2} \\ &= g^{\frac{bU_2 + aU_1}{U_1 U_2}} = g^{\frac{\text{GCD}(U_1, U_2)}{U_1 U_2}} = g^{1/\text{LCM}(U_1, U_2)} \end{aligned}$$

Here, a, b are integers such that $aU_1 + bU_2 = \text{GCD}(U_1, U_2)$, and they can be found in polynomial time using the extended Euclid algorithm. Due to Bezout's identity a, b always exist.

- **Difference of two sets:** We define the set difference operator $\ominus$ below

$$\sigma_{S_1} \ominus_{pk} S_2 = \sigma_{S_1 \setminus S_2} = \sigma_{S_1}^{U_2} = g^{U_2/U_1}$$

, where $S_2 \subseteq S_1$,

- **Homomorphic property over randomization:** The set difference operator $\ominus$ is homomorphic over the randomization function.

$$\begin{aligned} f_{pk}(\sigma_{S_1}, r) \ominus_{pk} S_2 &= g^{r \cdot U_2/U_1} = \\ f_{pk}(g^{U_2/U_1}, r) &= f_{pk}(\sigma_{S_1} \ominus_{pk} S_2, r) \end{aligned}$$

We show in App. E that the security of the scheme reduces to the strong QR-RSA assumption in the random oracle model.

Distributed Key Generation & Signing: To leverage the signature scheme with multiple signers, as required to publish an ACB, we need to extend it to a distributed setting such that the auditors only have a share of the private key. Additionally, the distributed key generation and sharing must be robust against a malicious minority of signers. Fortunately, our signature scheme can be extended to support distributed key generation and signing using existing techniques. Due to space constraints, we define the ideal functionality $\mathcal{F}_{\text{dsign}}$ for distributed key generating and signing, and describe the protocol realizing the functionality in App. B.

5.2.3 Blocklist construction. We can create a signature based blocklist using a randomizable set-homomorphic signature as follows (see Fig. 5)

Algorithm 2 Homomorphic Signature Based Blocklisted

```

1: procedure Publish( $\mathcal{B}$ )
2:   Auditors jointly sample a key pair for by invoking  $\mathcal{F}_{\text{dsign}}, \langle pk^{(r)}, sk^{(r)} \rangle$ ,
   where the private key is secret-shared
3:   Auditor  $A_k$  samples a key pair for a standard digital signature scheme
    $\langle pk_k, sk_k \rangle$ 
4:   Auditors invoke  $\mathcal{F}_{\text{dsign}}$  to sign the blocklist  $\sigma_{\mathcal{B}} \leftarrow \text{SIGN}_{sk}(\mathcal{B})$  and publish
   the ACB  $\mathcal{T}_{\text{sig}} \leftarrow \langle pk^{(r)}, \sigma_{\mathcal{B}} \rangle$ 
5:   Auditors sign the ACB under their own private keys  $\Sigma_k \leftarrow$ 
    $\text{SSIGN}_{sk_k}(h(\mathcal{T}_{\text{sig}}))$  and publish.
6:   CLIENT downloads  $\mathcal{T}_{\text{sig}}$  and verifies the signature under all the auditors' public
   keys  $\{pk_k\}_{k \in [1, \eta]}$ .

7: procedure Search( $y, \mathcal{T}_{\text{sig}}$ )
8:   CLIENT computes
    $\sigma_{\mathcal{B} \setminus \{y\}} \leftarrow \sigma_{\mathcal{B}} \ominus_{pk^{(r)}} \{y\}$ 
    $\text{RAND}_{pk^{(r)}}(\sigma_{\mathcal{B} \setminus \{y\}}) = \langle v, w \rangle$ 
    $C_{\text{DATA}} \leftarrow \langle v, w \rangle$ 
   CLIENT sends  $C_{\text{DATA}}$  to the SERVER
9:   SERVER computes and outputs the result of  $\text{AUTH}_{pk^{(r)}}(\mathcal{B}, C_{\text{DATA}})$ .
```

Costs: The costs of creating and searching are as follows

- CreateBlocklist has $\mathcal{O}(n)$ cost for the auditors, client and server. This is a one-time cost
- Search has $\mathcal{O}(1)$ cost for the client and the server
- The total storage cost for the client is $\sum_{k \in [1, \zeta]} |bf_k|$

Security: The security of the ACB is due to the following

- Confidentiality and unforgeability is ensured by the security of $\mathcal{F}_{\text{authOPRF}}$, and the IND-CPA security of the encryption scheme.

5.3 Encrypted Cuckoo Hash Based Blocklist

The blocklist constructions we have considered so far do not support threshold searching. In this section, we will design a ACB that supports threshold search. The construction leverages a cuckoo hash table. Intuitively, the blocklisted items are hashed into a cuckoo hash table T of size $n\gamma(1 + \epsilon_{\text{CHT}})$ with hash functions h'_1, h'_2 , where ϵ_{CHT} is a parameter that controls false negative rate [8]. The hash table is stored on the client and the values in each location is individually encrypted under an additively homomorphic encryption scheme. The hash keys are not stored alongside: the client sees

Experiment $\text{EXPT}_{\Pi_{\text{Sig}}}^{\text{uf-cma}}(A_{\text{ufcma}})$	Experiment $\text{EXPT}_{\Pi_{\text{Sig}}}^{\text{ind-sr}[b]}(A_{\text{indsr}}^* = (A_{\text{indsr}}[1], A_{\text{indsr}}[2], A_{\text{indsr}}[3]))$
$(pk, sk) \leftarrow \text{GEN}(1^\lambda)$ $(S, \sigma) \leftarrow A_{\text{ufcma}}^{\text{SIGN}_{sk}(\cdot)}(pk)$ if $\text{VER}_{pk}(S, \sigma) = 1 \wedge S \setminus \phi \neq \emptyset$ where ϕ is the union of all the sets queried to $\text{SIGN}_{sk}(\cdot)$ then output 1 else output 0	$(pk, sk) \leftarrow \text{GEN}(1^\lambda)$ $(S, \phi_1) \leftarrow A_{\text{indsr}}[1]^{\text{SIGN}_{sk}(\cdot)}(pk)$ $\sigma_S \leftarrow \text{SIGN}_{sk}(S)$ $(y_0, y_1, \phi_2) \leftarrow A_{\text{indsr}}[2]^{\text{SIGN}_{sk}(\cdot)}(\sigma, \phi_1)$ If $y_0 \in \phi \cup S \vee y_1 \in \phi \cup S$ where ϕ is the union of all the sets queried to $\text{SIGN}_{sk}(\cdot)$ then abort $b' \leftarrow A_{\text{indsr}}[3](\text{RAND}_{pk}(\sigma \ominus_{pk} \{y_b\}), \phi_2)$ If $b' = b$ then output 1 else output 0

Figure 3: Security experiments for a randomizable set homomorphic signature

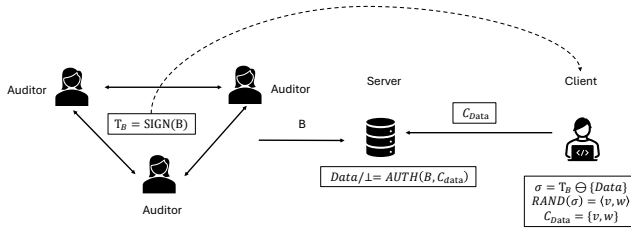


Figure 4: Randomizable set-homomorphic signature for blocklist construction. The auditors sign the blocklist. During an search, the client homomorphically computes a signature on $B \setminus \{\text{DATA}\}$ and randomizes it. The server obtains the client's data when $\text{DATA} \in B$. (The keys for the signature have been omitted for simplicity).

$\Pi_{\text{RH-Sig}}$: Randomizable Set Homomorphic Signature

Parameters: $H : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ and $H_P : \mathcal{U} \rightarrow \mathcal{P}$ are both random oracle, and $\mathcal{P}$ is the set of odd primes less than a semi-prime modulus N .

- $\text{GEN}(1^\lambda)$: Select two safe primes of length λ -bits $p = 2p' + 1$ and $q = 2q' + 1$ and compute $N = pq$. Publish N , and a generator g of QR_N as the public key. The order of QR_N is the private key.
- $\text{SIGN}_{sk}(S)$: Compute $U \leftarrow \prod_{y \in S} H_P(y)$, and output signature $\sigma_S \leftarrow g^{1/U} \bmod N$.
- $\text{VER}_{pk}(S, \sigma_S)$: Compute $U \leftarrow \prod_{y \in S} H_P(y)$ and output 1 if $\sigma_S^U \equiv g \bmod N$. Otherwise, output 0.
- $\text{RAND}_{pk}(\sigma_S)$: Sample $r \leftarrow [1, N^2]$. Output $v = f_{pk}(\sigma_S, r) = \sigma_S^r \bmod N$ and $w = H(g^r \bmod N)$.
- $\text{AUTH}_{pk}(S, (v, w))$: For every $v \in S$, compute $U \leftarrow \prod_{y \in S \setminus \{v\}} y$ and output v if $H(v^U \bmod N) = w$. Otherwise, output $\perp$.

Figure 5: Randomizable set homomorphic signature under the strong QR-RSA assumption

$2n\gamma(1 + \epsilon_{\text{CHT}})$ encrypted values, and knows the relation between ciphertexts and their corresponding locations in the table. The empty locations in the table are padded with encryptions of dummy values.

During a threshold search the client performs lookups in T , and then compares its inputs with the encrypted values in the corresponding locations in the hash table, by homomorphically subtracting one from the other, and blinding the result with a random multiplicative factor. If the projections match, then after decryption,

the server should observe an encryption of 0. To support threshold searches, the client secret-shares a known value in the encrypted results provided to the server. After decryption, if the server obtains enough shares, then it can reconstruct the value and confirm. The main challenge is

5.3.1 Detectable Secret Sharing. Similar to standard (ℓ_{ss}, t_{ss}) secret-sharing, a detectable secret sharing (DSS) scheme has two algorithm $\text{dsshare} = \langle \text{DSHARE}, \text{DCOMBINE} \rangle$.

- $\{ss_1, \dots, ss_{\ell_{ss}}\} \leftarrow \text{DSHARE}(\kappa)$ takes as input a secret κ and it produces a set of secret-shares $\{ss_1, \dots, ss_{\ell_{ss}}\}$
- $\kappa/\perp \leftarrow \Pi_{\text{dss}}.\text{DCOMBINE}(\{x_1, \dots, x_{\ell_{ss}}\})$ Given a set of ℓ_{ss} elements, if at least t_{ss} of them are valid secret shares of the same secret, and the rest are random elements, output the secret κ . Otherwise, output $\perp$.

For security Π_{dss} ensures that when there are less than t_{ss} number of shares in $\{x_1, \dots, x_{\ell_{ss}}\}$ then nothing is revealed to an adversary that has the set *including which of the shares are correct*.

Detectable hash function: We will make use of a primitive introduced by Bhowmick et al. [8] called a detectable hash function to realize a DSS scheme. Formally, a (t_{dhf}, s_{dhf}) -detectable hash function $\text{DHF} : K_{dhf} \times \mathcal{D} \rightarrow \mathcal{R}$ has the following two properties.

- (1) The function is weakly t_{dhf} -wise independent, that is the distribution of $\langle \text{DHF}_{k_{dhf}}(x_1), \dots, \text{DHF}_{k_{dhf}}(x_{t_{dhf}}) \rangle$ given $k_{dhf} \leftarrow K_{dhf}$ and $x_1, \dots, x_{t_{dhf}} \leftarrow \mathcal{D}$ is identical to the uniform distribution over $\mathcal{D}^{t_{dhf}}$.
- (2) The function is s_{dhf} -detectable. Given a vector $\vec{z} = \langle z_1, \dots, z_{\ell_{dhf}} \rangle \in \mathcal{D}^{\ell_{dhf}}$, where for $I \subseteq [\ell_{dhf}]$, $z_j \leftarrow \text{DHF}_{k_{dhf}}(x_j)$ when $j \in I$, and $z_j \leftarrow \mathcal{D}$ when $j \notin I$, there is an algorithm DETECT
 - $\text{DHF}_{k_{dhf}}.\text{DETECT}(\vec{z}) = I$ with non-negligible probability, when $|I| \geq \max(t_{dhf} + 1, \ell - s_{dhf})$.
 - $\text{DHF}_{k_{dhf}}.\text{DETECT}(\vec{z}) = \text{Fail}$ when $|I| \leq t_{dhf}$.

Intuitively, given a vector $\vec{z} \in \mathcal{D}^{\ell_{dhf}}$, where at least $\max(t_{dhf} + 1, \ell_{dhf} - s_{dhf})$ components are outputs of DHF, the detection algorithm identifies these components. However, if less than $t_{dhf} + 1$ entries are generated by DHF, then the detection algorithm does not reveal any information about I .

Protocol for detectable secret sharing: The protocol for a DSS is as follows.

Algorithm 3 dsshare: detectable secret sharing scheme

```

1:  $\triangleright$  Secret sharing a value  $\kappa$  into  $\ell_{ss}$  shares  $\triangleleft$ 
2: procedure SHARE( $\kappa$ )
3:   Secret-shares  $\kappa$  into  $\ell_{ss}$  shares using a  $(\ell_{ss}, t_{ss})$  secret sharing scheme
   sshare  $\leftarrow$  (SHARE, COMBINE):  $\{ss_1, \dots, ss_{\ell_{ss}}\} \leftarrow$  SHARE( $\kappa$ )
4:   The dealer samples  $\ell_{ss}$  one-time pads,  $\kappa_1, \dots, \kappa_{\ell_{ss}}$  and computes the set
    $SS \leftarrow \{(\kappa_1, ss_1), \dots, (\kappa_{\ell_{ss}}, ss_{\ell_{ss}})\}$ 
5:   The dealer samples a key for a  $(\ell_{ss} - t_{ss}, t_{ss} - 1)$ -detectable hash function
    $k_{dhf}$ , and computes the set
    $D \leftarrow \{DHF_{k_{dhf}}(r_1) \oplus \kappa_1, \dots, DHF_{k_{dhf}}(r_n) \oplus \kappa_n\}$ 
   , where  $r_1, \dots, r_{\ell_{ss}} \leftarrow \mathcal{D}$ .
6:   return SS, D
7:  $\triangleright$  Reconstructing the detectable secret shares  $\triangleleft$ 
8: procedure RECONSTRUCT( $SS' \leftarrow \{(\kappa'_1, ss'_1), \dots, (\kappa'_{\ell_{ss}}, ss'_{\ell_{ss}})\}, D$ )
9:   Compute the vector  $\tilde{z} \leftarrow (\kappa'_i \oplus d_i)_{i \in [1, \ell_{ss}]}$ 
10:  Compute  $DHF_{k_{dhf}} \cdot \text{DETECT}(\tilde{z}) = \mathcal{I}$ . If the algorithm fails, then abort. Otherwise,
   construct  $\mathcal{S} \leftarrow \{ss'_i : i \in \mathcal{I}\}$  and reconstruct the secret shares
    $\kappa \leftarrow$  COMBINE( $\mathcal{S}$ )
11: return  $\kappa/\perp$ 

```

Algorithm 4 Encrypted Hash Table Based Blocklist

```

1: procedure Publish( $\mathcal{B}$ )
2:   Auditor  $A_k$  samples a key pair for a standard digital signature scheme
    $\langle pk_k, sk_k \rangle$ 
3:   The auditors initialize a cuckoo hash table  $T$  with  $(1 + \epsilon_{\text{CHT}}) \cdot n$  slots, and
   samples two hash function  $h'_1, h'_2 : \{0, 1\}^\lambda \rightarrow [(1 + \epsilon_{\text{CHT}}) \cdot n]$ .
4:   The auditors sample a key pair  $\langle pk_e, sk_e \rangle$  for an additively homomorphic
   encryption scheme  $\Pi_{\text{ahe}} = \langle \text{AH\_KEYGEN}, \text{AH\_ENC}, \text{AH\_DEC} \rangle$ .
5:   For  $b \in \mathcal{B}$ , the auditors place  $\text{AH\_ENC}_{pk_e}(b)$  at slot  $T[h'_1(\pi_j(b))]$  or
    $T[h'_2(\pi_j(b))]$  using cuckoo hashing. The ACB is .
6:   The auditors sign the ACB  $\mathcal{T}_{\mathcal{B}} \leftarrow T$  under their own private keys and publish
    $\{\text{SIGN}_{sk_k}(h(\mathcal{T}_{\mathcal{B}}))\}_{k \in [1, \eta]}$ 
7: procedure TSearch( $Y, \mathcal{T}_{\text{CHT}}, t$ )
8:    $\triangleright$  Client secret-shares a secret value and encrypts the shares  $\triangleleft$ 
9:   CLIENT invokes a  $(2\ell, t)$ -DSS scheme dsshare to obtain  $\langle SS, D \rangle \leftarrow$ 
   DSHARE( $0$ ). Let  $SS = \{s_1^*, \dots, s_{2\ell}^*\}$ 
10:  for  $y_j \in y_1, \dots, y_\ell$  do
11:    CLIENT retrieves  $c_1$  from  $T[h'_1(y_j)]$  and  $c_2$  from  $T[h'_2(y_j)]$  and computes
    $u_{1,j} \leftarrow \text{Enc}_{pk_e}(r_{1,j} \times (c_1 - y_j) + s_j^*)$ 
    $u_{2,j} \leftarrow \text{Enc}_{pk_e}(r_{2,j} \times (c_2 - y_j) + s_{\ell+j}^*)$ 
   where  $r_{1,j}, r_{2,j} \leftarrow \mathcal{S} [1, q - 1]$ .
12:  CLIENT sends to SERVER  $\{u_{i,j}\}_{i \in [2], j \in [1, \ell]}$ 
13:   $\triangleright$  Upon receiving  $\{u_{i,j}\}_{i \in [2], j \in [1, \ell]}$  from CLIENT  $\triangleleft$ 
14:  SERVER decrypts  $u_{i,j}$  for all  $i \in [1, 2]$  and  $j \in [1, \ell]$ , and obtains  $\{s'_1, \dots, s'_{2\ell}\}$ .
   If  $\Pi_{dss} \cdot \text{COMBINE}(\{s'_1, \dots, s'_{2\ell}\}) = \perp$ , then SERVER outputs 0. Otherwise it
   outputs 1.

```

5.3.2 Blocklist construction.

6 Content Moderation Schemes

In this section, we will describe the content moderation schemes realizing $\mathcal{F}_{\text{ExactMod}}$ and $\mathcal{F}_{\text{FuzzyMod}}$ leveraging the blocklist constructions described in Sec. 5.

6.1 Exact Matching

We describe our content moderation scheme with exact-matching capabilities in Fig. ??; we assume that the client wants to upload files that are drawn from the universe $\mathcal{U}$, and the content hash function $embed : \mathcal{U} \rightarrow \{0, 1\}^\ell$ maps any such content to an ℓ -bit string. Two items u, u' are similar iff. $embed(u) = embed(u')$.

Key idea: fast path with hybrid verification: We can directly leverage either a Bloom filter based blocklist or a signature based blocklist to realize $\mathcal{F}_{\text{ExactMod}}$. In the former, the cost of matching against the blocklist is $\mathcal{O}(1)$ but the construction has false positives. On the other hand, the signature based construction does not have false positives (or false negatives) but linear (in the blocklist size) matching costs for the server. This motivates the questions: *can we design a scheme that has sublinear matching costs without incurring false positives?* Our solution is to combine the two aforementioned ACB constructions as follows ().

Creating the Blocklist:

- (1) The auditors partition the blocklisted items into τ disjoint partitions $\mathcal{B}^{(1)}, \dots, \mathcal{B}^{(\tau)}$.
- (2) The auditors invoke $\Pi_{\text{sig}} \cdot \text{Publish}(\mathcal{B}^{(1)}, \dots, \mathcal{B}^{(\tau)})$ and publish a separate signature-based ACB for each of the blocklist partitions $\mathcal{T}_{\text{Sig}}^{(1)}, \dots, \mathcal{T}_{\text{Sig}}^{(\tau)}$.
- (3) The auditors invoke $\Pi_{bf} \cdot \text{Publish}(\mathcal{B})$ and publish a blinded blocklist $\mathcal{B}^{(\text{blind})}$.
- (4) The client and server invoke $\Pi_{bf} \cdot \text{Preprocess}(\mathcal{B}^{(\text{blind})})$ and the client learns $\mathcal{T}_{\text{BF}}$.

Upload: During a file upload with DATA

- (1) (*fast path*.) The client derives the content hash $y \leftarrow embed(\text{DATA})$ and invokes $\Pi_{bf} \cdot \text{Search}(y, \mathcal{T}_{\text{BF}})$. If the search returns 0, then the client and server can abort because this means $y \notin \mathcal{B}$. The server will store the encrypted content.
- (2) (*fine-grained check*.) Otherwise, let bf_k be the Bloom filter such that $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(y)) \in bf_k$; recall that the server learns bf_k in $\Pi_{bf} \cdot \text{Search}$. In this case, the server identifies the blocklist partitions where the items included in bf_k . More specifically, the server calculates the set of indices

$$\mathcal{I} \leftarrow \{m : b \in bf_k \wedge b \in \text{Part}_m\}$$

, and sends it to the client

- (3) Upon receiving $\mathcal{I}$, the client invokes $\Pi_{\text{sig}} \cdot \text{Search}(y, \mathcal{T}_{\text{Sig}}^{(m)})$ for all $m \in \mathcal{I}$. If any of these invocations return y to the server, then $y \in \mathcal{B}$. In this case, the server rejects the encrypted content and aborts. Otherwise, $y \notin \mathcal{B}$ and the server accepts the content.

Costs:

- The preprocessing for $\mathcal{T}_{\text{BF}}$ requires a one-time $\mathcal{O}(n)$ compute costs and communication for the client and the server. The client also requires $\mathcal{O}(n^{1/\epsilon_{bf}} + \tau)$ storage. During uploads, the client and server costs are as follows
- $\mathcal{O}(1)$ compute cost for the client and server (with high probability) when the client's uploaded file does not match an item in the blocklist, i.e., fast path verification.
- $\mathcal{O}(\log n)$ compute cost for the client and $\mathcal{O}(n)$ server cost when the client's uploaded file matches an item in the blocklist
- $\mathcal{O}((n/\tau) + \log n)$ communication cost for sending an encrypted Bloom filter, and $\mathcal{O}(\log n)$ content verifications tokens if a fine-grained check is deemed necessary. If $\tau \in \mathcal{O}(n)$, then the cost of an upload is $\mathcal{O}(\log n)$.

Security:

Algorithm 5 Π_{EM} : content moderation with exact matching

```

1:  $\triangleright$  Distribute blocklist into partitions and publish signature-based ACB  $\triangleleft$ 
2: Auditors distribute  $\mathcal{B}$  into  $\tau$  disjoint buckets  $\mathcal{B}^{(1)}, \dots, \mathcal{B}^{(\tau)}$ 
3: for each  $m \in [1, \tau]$  do
4:   Auditors invoke  $\Pi_{sig}.Publish(\mathcal{B}^{(m)})$  and publish the signature-based ACB
5:    $\mathcal{T}_{Sig}^{(m)} = \langle pk^{(r)}, \sigma_{\mathcal{B}^{(m)}} \rangle$ 
6:   Auditors sign and publish  $\{SSIGN_{sk_k}(h(\mathcal{T}_{Sig}^{(m)}))\}_{k \in [1, \eta]}$ 
7:  $\triangleright$  Publish blinded blocklist for Bloom filter based ACB  $\triangleleft$ 
8: Auditors invoke  $\Pi_{bf}.Publish(\mathcal{B})$  and publish a blinded blocklist  $\mathcal{B}^{(blind)}$  and
   the signatures  $\{SSIGN_{sk_k}(h(\mathcal{B}^{(blind)}))\}_{k \in [1, \eta]}$ 
9:  $\triangleright$  Client and server preprocess blinded blocklist  $\triangleleft$ 
10: CLIENT downloads  $\mathcal{B}^{(blind)}$ ,  $\{SSIGN_{sk_k}(h(\mathcal{B}^{(blind)}))\}_{k \in [1, \eta]}$ ,  $\{\mathcal{T}_{Sig}^{(m)}\}_{m \in [1, \tau]}$ ,
     $\{\{SSIGN_{sk_k}(h(\mathcal{T}_{Sig}^{(m)}))\}_{k \in [1, \eta]}\}_{m \in [1, \tau]}$ , and verifies the signatures. If the sig-
    natures do not verify, then CLIENT aborts.
11: CLIENT and SERVER invoke  $\Pi_{bf}.Preprocess(\mathcal{B}^{(blind)})$ . CLIENT stores  $\mathcal{T}_{BF} \leftarrow$ 
     $\{ENC_k(bf_1), \dots, ENC_k(bf_\ell)\}$ 
12:  $\triangleright$  Matching against blocklist during uploads  $\triangleleft$ 
13: CLIENT computes  $y \leftarrow embed(DATA)$  and  $\Pi_{bf}.Search(y, \mathcal{T}_{BF})$ .
14: if  $\Pi_{bf}.Search(y, \mathcal{T}_{BF}) = 0$  then  $\triangleright$  Fast path
15:   SERVER outputs  $\{ID, detected = FALSE, \emptyset\}$  and aborts.
16: else  $\triangleright$  Fine-grained check
17:   SERVER computes  $\mathcal{I} \leftarrow \{m : b \in bf_k \wedge b \in Part_m\}$  and sends to CLIENT.
18:   for each  $m \in \mathcal{I}$  do
19:     CLIENT invokes  $\Pi_{sig}.Search(y, \mathcal{T}_{Sig}^{(m)})$ 
20:     if  $\Pi_{sig}.Search(y, \mathcal{T}_{Sig}^{(m)}) = y$  then
21:       SERVER outputs  $\{ID, detected = TRUE, y\}$  and aborts
22:     else
23:       SERVER outputs  $\{ID, detected = FALSE, \emptyset\}$ . CLIENT adds  $y$  to  $\mathcal{L}$ .

```

6.2 Fuzzy Matching

we will describe a content moderation scheme with fuzzy matching that realizes $\mathcal{F}_{FuzzyMod}$. We will assume that the set of blocklisted items are embeddings in a metric space with distance d that produces a ℓ -dimensional vector, and we will refer to the blocklisted items as vectors henceforth. Our protocol will detect matches of the form $\langle \vec{b}, \vec{y} \rangle$ such that $\|b - y\|_d \leq t$. In return, the protocol will have a small number of false negatives – cases where items are blocked but are not detected.

Key Idea: We will use projections of the blocklist vectors to perform a *fast path* check with low false positives, and if required, follow it by performing a significantly more expensive but perfectly accurate test over the actual vectors. Specifically, if the client's input matches a certain threshold number of projections against the blocklisted items, then the server performs the fine-grained check. Obviously leaking the projections themselves will allow the server to learn information about the encrypted content. Therefore, the fast path only reveals the number of matching projections, and the likely candidates in the blocklist that should be checked against the input. We note that the $\mathcal{F}_{Mod}$ functionality allows this leakage, since the server does not learn the content key. Nonetheless, we select projection functions carefully such that i) the chance of false positives are low, ii) the projection functions are themselves randomly chosen, and iii) the candidate set is reasonably large and can be set as a tunable parameter. A caveat here is that the fine-grained check must be performed over the actual embedding vectors and not just the projections, so the blocklist authentication token and

the content verification token must be modified to account for entire vectors. To realize this idea, we will first define the projection functions, and then describe how to modify the tokens.

6.2.1 Locality-Preserving Fingerprints.

Definition 6.1. A locality-preserving fingerprint of an element m in the set $\mathcal{B} \subseteq \mathcal{M}$ comprises its projections under uniformly sampled functions $\pi_1, \dots, \pi_\gamma : \mathcal{M} \rightarrow \mathcal{S}$, $FP(m) \leftarrow \{h(\pi_1(m), \pi_1) \dots, h(\pi_\gamma(m), \pi_\gamma)\}$, where $h : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a collision-resistant hash function, such that given any $m' \in \mathcal{M}$,

$$\mathbb{P}(\nexists j \in [1, \gamma], \pi_j(m) = \pi_j(m') \mid \|m - m'\|_d \leq t) \leq \epsilon_{fpr}^\gamma \quad (6)$$

$$\mathbb{P}(\nexists j \in [1, \gamma], \pi_j(m) \neq \pi_j(m') \mid \|m - m'\|_d > t) \leq \epsilon_{fpr}^\gamma \quad (7)$$

where, ϵ_{fpr} and ϵ_{fnr} are the false positive rate and false negative rate under each projection function.

Intuitively, Eq. (6) ensures that two items close in the metric space agree on at least one of the projections with high probability. Eq. (7) implies that two items far apart in the metric space do not agree on all the projections with high probability. Note that ϵ_{fpr} and ϵ_{fnr} may not be negligible. Also, since the projection functions are selected randomly, the overall false positives and false negatives over the projections are binomially distributed

COROLLARY 6.2. Let $t_D \in [1, \gamma]$. Given a dataset $\mathcal{B}$ of n elements, all having unique projections under γ random projection functions $\pi_1, \dots, \pi_\gamma$, and a test point m . The probability that for a subset of projection functions I , where $|I| \geq t_D$, the probability that $\pi_j(m) = \pi_j(m')$ for some $m' \in \mathcal{B}$ and $\|m - m'\|_d > t$ is bounded by

$$\mathbb{P}(\text{BAD}_{FP}) \leq \sum_{i=t_D}^{\gamma} \binom{\gamma}{i} \epsilon_{fpr}^i (1 - \epsilon_{fpr})^{\gamma-i}$$

, where $\mathcal{M}$ is a metric space with distance $\|\cdot\|_d$, ϵ_{fpr} is the false positive rate under each random projection.

COROLLARY 6.3. Let $t_D \in [1, \gamma]$ and two elements $m, m' \in \mathcal{M}$ such that $\|m - m'\|_H \leq t$. Then the probability that they do not agree on at least $\gamma - t_D + 1$ projections is bounded by

$$\mathbb{P}(\text{BAD}_{FN}) \leq 1 - \sum_{i=0}^{\gamma-t_D} \binom{\gamma}{i} \epsilon_{fnr}^i (1 - \epsilon_{fnr})^{\gamma-i}$$

, where ϵ_{fnr} is the false negative rate under each random projection.

COROLLARY 6.4. Let $t_D \in [1, \gamma]$. Given two elements $m, m' \in \mathcal{M}$,

$$\mathbb{P}(|FP(m) \cap FP(m')| < \gamma - t_D + 1 \mid \|m - m'\|_d \leq t) \leq \mathbb{P}(\text{BAD}_{FN})$$

$$\mathbb{P}(|FP(m) \cap FP(m')| \geq t_D \mid \|m - m'\|_d > t) \leq \mathbb{P}(\text{BAD}_{FP})$$

Locality-preserving fingerprint for the Hamming distance: Chongchitmate et al. [13] describe a construction for a locality-preserving fingerprint for the Hamming space using uniformly sampled functions $\pi_1, \dots, \pi_\gamma$ where $\pi_j : \{0, 1\}^\ell \rightarrow \{0, 1\}^\lambda$. Each function specifies a random subset of bits from the input vector, and injectively maps it to a λ -bit string. Consider a set $\mathcal{B} \subseteq \{0, 1\}^\ell$.

DB size	Our protocol Π_{FM}^*		PAMC [26, Table 8]	
	Time (in sec.)	Comm. (in KB)	Time (in sec.)	Comm. (in KB)
1048576	0.59 (61)	20 (1310)	27.5	508
2097152	0.59 (61)	20 (1310)	27.5	586
4194304	0.59 (61)	20 (1310)	27.7	742
8388608	0.59 (61)	20 (1310)	28.3	1053.9

Table 2: Comparison of Π_{FM}^* against PAMC. The time inside the parentheses is the time for a fine-grained check, if required.

- (1) Create γ key-value stores $S_1, \dots, S_\gamma$ where the keys are the vectors in $\mathcal{B}$.
- (2) For each function π_j and each element $\vec{b} \in \mathcal{B}$
 - (a) Evaluate $h(\pi_j(\vec{b}), j)$ where $h : \{0, 1\}^{\lambda + \log \gamma} \rightarrow \{0, 1\}^\lambda$ is a collision-resistant hash function.
 - (b) Insert $h(\pi_j(\vec{b}), j)$ in the key-value store S_j at index $S_j[\vec{b}]$ unless $\pi_j(\vec{b})$ is already there in S_j . If so, assign a random value from the range $\{0, 1\}^\lambda$.

The set $FP_{\vec{b}} \leftarrow \{S_1[\vec{b}], \dots, S_\gamma[\vec{b}]\}$ is a locality-preserving fingerprint of $\vec{b}$ assuming negligible collisions in h . If all the vectors in $\mathcal{B}$ are at least ct distance apart, then $\gamma = \mathcal{O}\left((nc)^{\frac{1}{c-1}}(\lambda + \log n)\right)$ projections ensure that $\epsilon_{\text{fpr}}^\gamma$ and $\epsilon_{\text{fmr}}^\gamma$ are both negligible in λ ; here $|\mathcal{B}| = n$. Observe that if $c > 2$, the number of projections are sub-linear in n . Most embedding functions ensure that similar items are close in the embedded space, while dissimilar items are sufficiently far apart [26, 29]. Alternatively, nearby items can be repeatedly clustered to have an efficient construction [13].

6.2.2 Protocol design. A locality-preserving fingerprint provides a straightforward construction realizing $\mathcal{F}_{\text{FuzzyMod}}$ (Algorithm. 6).

- (1) The auditors publish an encrypted cuckoo hash table based blocklist by invoking $\Pi_{\text{cht}}.\text{Publish}$ using the locality-preserving fingerprints of all the blocklist vectors.
- (2) During an upload,
 - (a) The client computes the content embedding $\vec{y} \leftarrow \text{embed}(\text{DATA})$.
 - (b) The client and server invoke $\Pi_{\text{cht}}.\text{TSearch}(FP(\vec{y}), \mathcal{T}_{\text{CHT}}, t_D)$ to perform a threshold search such that the search returns a 1 when $|FP(\vec{y}) \cap \mathcal{T}_{\text{CHT}}| \geq t_D$. Due to Corollary. 6.4 this implies that there is some vector $\vec{b} \in \mathcal{B}$ such that $\|\vec{y} - \vec{b}\|_H \leq t$ with probability at least $1 - \mathbb{P}(\text{BAD}_{FP})$.
 - (c) When $\Pi_{\text{cht}}.\text{TSearch}(FP(\vec{y}), \mathcal{T}_{\text{CHT}}, t_D) = 1$, the server adds the content ID to an *flagged content list* $\mathcal{D}$.
 - (d) If there are already a predefined threshold t_R number of content IDs in $\mathcal{D}$, then the server deletes all the corresponding content and informs the client. Otherwise, the server accepts the encrypted content and stores it.

We observe that

7 Evaluation

All our protocols have been implemented and benchmarked⁴. We implemented the protocols in C++ and Python. Group arithmetic was implemented using the NTL library⁵. The gRPC library⁶ was

⁴<https://anonymous.4open.science/r/content-mod-code-2125/>

⁵<https://libntl.org>

⁶<https://grpc.io>

Algorithm 6 Π_{FM} : content moderation with exact matching

```

1:  $\triangleright$  Create a encrypted CHT based blocklist using locality-preserving fingerprints  $\triangleleft$ 
2: for  $\vec{b} \in \mathcal{B}$  do
3:   Auditors compute the the locality-preserving fingerprint  $FP_{\vec{b}}$ 
4:   Auditors invoke  $\Pi_{\text{cht}}.\text{Publish}\left(\{FP_{\vec{b}}\}_{\vec{b} \in \mathcal{B}}\right)$  and publish  $\mathcal{T}_{\text{CHT}}$  and
      $\{\text{SSIGN}_{sk_k}(h(\mathcal{T}_{\vec{b}}))\}_{k \in [1, \eta]}$ .
5:  $\triangleright$  Threshold matching against blocklist during uploads  $\triangleleft$ 
6: CLIENT sends  $(ID, \text{ENCK}_f(\text{DATA}))$  to SERVER
7: CLIENT computes  $\vec{y} \leftarrow \text{embed}(\text{DATA})$  and  $FP_{\vec{y}}$ .
8: CLIENT and SERVER invoke  $\Pi_{\text{cht}}.\text{TSearch}(FP_{\vec{y}}, \mathcal{T}_{\text{CHT}})$ .
9: if  $\Pi_{\text{cht}}.\text{TSearch}(FP_{\vec{y}}, \mathcal{T}_{\text{CHT}}) = 1$  then
10:   SERVER adds ID to  $\mathcal{D}$ 
11:   if  $|\mathcal{D}| \geq t_R$  then
12:     SERVER deletes all files with ID  $\in \mathcal{D}$ 
13:   else
14:     SERVER stores the encrypted file.
15:   SERVER outputs  $\langle ID, \text{detected} = \text{TRUE} \rangle$ .
16: else
17:   SERVER outputs  $\langle ID, \text{detected} = \text{FALSE} \rangle$ 

```

used for implementing communication between the client and the server. For the OPRF and the partially homomorphic encryption scheme, we used the *oblivious* library⁷, and the *lightPHE* library [40] respectively. We used the exponential Elgamal encryption scheme for the additively-homomorphic encryption. All parameters were configured to provide 112 bits of security. Benchmarks were performed with a single server running on an Amazon t2.xlarge instance with 8vCPUs and 32GB of memory. Our implementations are single-threaded. Clients were run on a t2.large instance with 2vCPUs and 8GB of memory.

7.1 Exact Matching

We conducted experiments for Π_{EM} using blocklists containing random SHA256 hashes. All tests were run 10 times and results were averaged across all runs. We compared our protocol with the PEMC protocol [26]⁸. *We stress that our protocol works against a malicious server in contrast to the semi-honest server in case of PEMC.*

Publishing the blocklist: Fig. 6a shows the compute costs for signing and publishing the blocklist. We set the number of partitions τ such that each partition has at most 64 items. This was fixed based on the time to verify a signature. Signing a blocklist with a million items takes roughly 170 seconds with up to 10 auditors

Preprocessing: Fig. 6b shows the compute time required for preprocessing the blocklist. Recall that this is a $\mathcal{O}(n)$ operation for both the client and the server. In our implementation, the OPRF of choice is the 2HashDH over curve25519⁹. The number of bins in our implementation was fixed to $n/\log n$. For a blocklist with a million elements, preprocessing takes roughly 21 seconds for the client and 36 seconds for the server. The encrypted Bloom filters require less than 4MB of client storage.

Uploads: We benchmarked the upload time in Π_{EM} by varying the fraction of queries that require a fast path verification (Fig. 6c). The reported times are averaged over 1000 queries. In the most

⁷<https://pypi.org/project/oblivious/>

⁸Their implementation is not publicly available but the paper reports benchmarks on a 6-core system with 32GB memory. For a fair comparison, we benchmarked our protocols on a similarly-provisioned system.

⁹<https://www.ietf.org/archive/id/draft-irtf-cfrg-ristretto255-00.html>

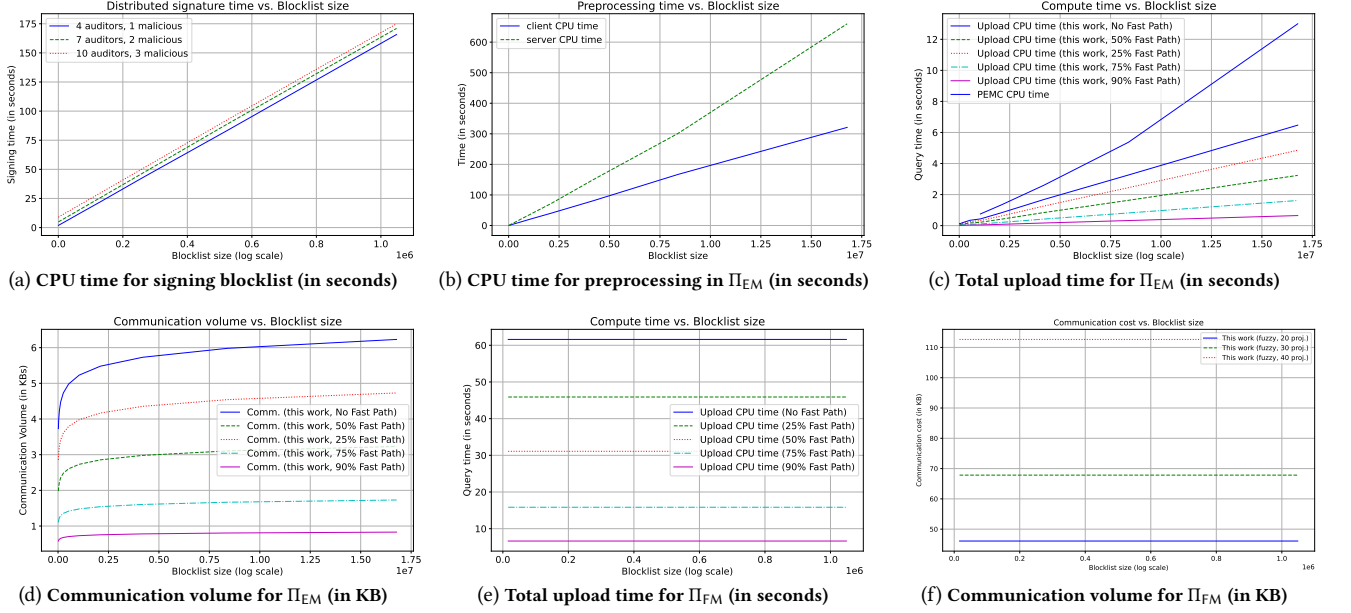


Figure 6: Costs for content moderation with exact and fuzzy matching

conservative case, where every upload results in a fine-grained check, the protocol requires roughly 430 milliseconds for a blocklist with a million items. In contrast, the PEMC protocol requires 750 milliseconds to perform the check. Moreover, when the fraction of queries served through a fast path verification increases, the compute times reduce proportionally. This feature is absent from the PEMC protocol, which runs a full check for every uploaded file. In fact, if we make a reasonable assumption that for a server detecting CSAM, a large fraction (90%) of the content will not be blocklisted, Π_{EM} achieves verification times of 43 ms per upload on average, a **12 $\times$ compute cost improvement over PEMC**.

Fig. 6d shows the communication costs of Π_{EM} . Recall that the communication involves a Bloom filter for a specific bin, which in our cases has $\log n$ items, and $\log n$ signatures when a fine grained check is required. Even in the most conservative case when none of the uploads are served by a fast path, the communication required in Π_{EM} for a 1 million-item blocklist is roughly 6KB. In comparison, PEMC requires 394KB since it performs a PIR over the blocklist. Thus, Π_{EM} has **50 $\times$ lower communication volume than PEMC**.

7.2 Fuzzy Matching

We conducted experiments for our fuzzy matching protocol Π_{FM} using a blocklist with random 256 bit vectors that simulate perceptual hashes. We instantiated the projection functions with the construction of Chongchitmate et al. [13]. The blocklist was partitioned into τ such that there are 64 vectors per bucket. We set the number of projections to 20 which gives a false positive rate $BAD_{FP} \leq 0.02$ and $BAD_{FP} \leq 0.07$. We compared against the private approximate matching (PAMC) protocol [26] that only tolerates a semi-honest server and has a higher false negative rate of roughly 17%

Uploads: Fig. 6e demonstrates the compute costs of Π_{FM} . In the conservative case where all uploads required a fine-grained check, Π_{FM} requires around 61 seconds per upload. *We recall that the cost of the protocol is independent of the blocklist size, and scales only in the number of projections and size of the vectors.* Increasing the fraction of uploads served by fast path checks, proportionally reduces the compute costs. When 90% of the uploads are served by the fast path (when blocklisted content is infrequently uploaded), the protocol requires less than 7 seconds per check on average.

We compared Π_{FM} against the PAMC protocol of Kulshrestha and Mayer [26], and Table 2 presents the results. We observe that in the average case, when the coarse-grained check does not identify a match, our protocol is faster (**by a factor of 5 $\times$**) and requires less communication (**by at least a factor of 25 $\times$**). However, the fine-grained check is more expensive.

8 Conclusion

This paper described protocols for detecting harmful content in encrypted files stored on an untrusted server, by comparing against a verified, common blocklist published by a distributed set of auditors. When computing exact hash matches against the blocklist, our protocol outperforms the state of the art both in terms of server and client costs. The protocol for fuzzy matching is secure against a malicious server, and features faster compute times on average than the state of the art that operate under a model with a semi-honest server.

9 Ethical Considerations

If deployed, our research will provide a tool for service providers to safeguard against misuse of their services to store harmful content. By introducing independent auditors, we reduce the potential risk of surveillance and censorship. However, the ethical risks of general content moderation itself needs to be taken into account when evaluating the ethical implications of our work. The debate regarding free speech vs. moderating misinformation does not have a conclusive answer but several works have tried to summarize arguments both in favor or against it [25].

Nonetheless, service providers have a responsibility to ensure that their services are not being misused to proliferate harmful content. Therefore, moderating for such content is justified, and arguably necessary [21]. However, to protect the interests of the clients, we need to ensure that auditors of the blocklist act independently and without influence. While in many cases this may not be possible, e.g., due to government interference, we believe this trust model is more suitable than endowing trust entirely on self-interested service providers.

All that said, our research itself, as a design for a content-moderation system, poses little ethical risk, at least directly. (The parameters and policies around its deployment need to be carefully considered, however.) Note, moreover, that our research did not involve testing with any real-world data or human subjects. As such, no direct harms or violations of privacy resulted from this research.

We acknowledge that, sometimes, the most ethical path is not to do the research or not to publish the research after it is complete. However, due to the urgent need to combat the proliferation of CSAM, we believe that the potential benefits of our research far outweigh its risks, thereby satisfying the Beneficence criterion of the Menlo Report.

References

- [1] 2025. Code repository for auditable content moderation. <https://anonymous.4open.science/r/content-mod-code-2125/README.md>
- [2] Joy Algesheimer, Jan Camenisch, and Victor Shoup. 2002. Efficient computation modulo a shared secret with application to the generation of shared safe-prime products. In *Advances in Cryptology – CRYPTO*. Springer, 417–432.
- [3] Inc. Amazon. 2023. An update on Amazon’s efforts to combat child sexual abuse material. <https://www.aboutamazon.com/news/policy-news-views/amazon-csam-transparency-report-2023>.
- [4] Apple, Inc. 2021. CSAM detection – Technical summary. https://www.apple.com/child-safety/pdf/CSAM_Detection_Technical_Summary.pdf.
- [5] Gilad Asharov and Yehuda Lindell. 2017. A full proof of the BGW protocol for perfectly secure multiparty computation. *Journal of Cryptology* 30, 1 (2017), 58–151.
- [6] BBC. 2022. Apple criticised for system that detects child abuse. <https://www.bbc.com/news/technology-58124495>.
- [7] M. Bellare, A. Desai, D. Pointcheval, and P. Rogaway. 1998. Relations among notions of security for public-key encryption schemes. In *Advances in Cryptology – CRYPTO 1998 (Lecture Notes in Computer Science, Vol. 1462)*.
- [8] Abhishek Bhowmick, Dan Boneh, Steve Myers, Kunal Talwar, and Karl Tarbe. 2021. The Apple PSI system. https://www.apple.com/child-safety/pdf/Apple_PSI_System_Security_Protocol_and_Analysis.pdf.
- [9] Ran Canetti and Gabriel Kapchuk. 2021. The Broken Promise of Apple’s Announced Forbidden-photo Reporting System – And How To Fix It. Center for Reliable Information Systems & Cyber Security, Boston University. <https://www.bu.edu/riscs/2021/08/10/apple-csam/>
- [10] Ran Canetti and Gabriel Kapchuk. 2026. The Broken Promise of Apple’s Announced Forbidden-photo Reporting System – And How To Fix It. <https://www.bu.edu/riscs/2021/08/10/apple-csam/>.
- [11] Dario Catalano, Rosario Gennaro, and Shai Halevi. 2000. Computing inverses over a shared secret modulus. In *Advances in Cryptology – EUROCRYPT*. Springer, 190–206.
- [12] Kevin Choi, Aathira Manoj, and Joseph Bonneau. 2023. Sok: Distributed randomness beacons. *IEEE*, 75–92.
- [13] Wutichai Chongchitmate, Steve Lu, and Rafail Ostrovsky. 2024. Approximate PSI with Near-Linear Communication. *Cryptology ePrint Archive* (2024).
- [14] CNN. 2022. Apple abandons controversial plan to check iOS devices and iCloud photos for child abuse imagery. <https://www.cnn.com/2022/12/08/tech/apple-csam-tool>.
- [15] S. Colombo, K. Nikitin, H. Corrigan-Gibbs, D. J. Wu, and B. Ford. 2023. Authenticated private information retrieval. In *32nd USENIX Security Symposium*.
- [16] Ronald Cramer and Victor Shoup. 2000. Signature schemes based on the strong RSA assumption. *ACM Transactions on Information and System Security* (2000).
- [17] Ling Du, Anthony TS Ho, and Runmin Cong. 2020. Perceptual hashing for image authentication: A survey. *Signal Processing: Image Communication* 81 (2020), 115713.
- [18] Dario Fiore, Rosario Gennaro, and Valerio Pastro. 2014. Efficiently verifiable computation on encrypted data. In *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*. 844–855.
- [19] National Center for Missing and Exploited Children. 2026. Missing Children Posters. <https://www.missingkids.org/home>.
- [20] Google, Inc. 2026. Fighting child sexual abuse online. <https://protectingchildren.google/introduction>.
- [21] Jeffrey William Howard. 2023. The ethics of social media: Why content moderation is a moral duty. *Journal of Practical Ethics* (2023).
- [22] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. 2014. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In *Advances in Cryptology – ASIACRYPT*. Springer, 233–253.
- [23] Patrick Jauernig, Ahmad-Reza Sadeghi, and Emmanuel Stappf. 2020. Trusted execution environments: properties, applications, and challenges. *IEEE Security & Privacy* 18, 2 (2020), 56–60.
- [24] Robert Johnson, David Molnar, Dawn Song, and David Wagner. 2002. Homomorphic signature schemes. In *Topics in Cryptology – CT-RSA 2002 (Lecture Notes in Computer Science, Vol. 2271)*. Springer, 244–262.
- [25] Anastasia Kozyreva, Stefan M Herzog, Stephan Lewandowsky, Ralph Hertwig, Philipp Lorenz-Spreen, Mark Leiser, and Jason Reifler. 2023. Resolving content moderation dilemmas between free speech and harmful misinformation. *Proceedings of the National Academy of Sciences* 120, 7 (2023), e2210666120.
- [26] Anunay Kulshrestha and Jonathan Mayer. 2021. Identifying harmful media in end-to-end encrypted communication: Efficient private membership computation. In *USENIX Security Symposium*. 893–910.
- [27] Diane Leblanc-Albarel and Bart Preneel. 2024. Black-box Collision Attacks on Widely Deployed Perceptual Hash Functions. *Cryptology ePrint Archive*, Paper 2024/1869. <https://eprint.iacr.org/2024/1869>
- [28] Jiwen Lu, Venice Erin Liong, and Jie Zhou. 2017. Deep hashing for scalable image search. *IEEE Transactions on Image Processing* 26, 5 (2017), 2352–2367.
- [29] Sean McKeown and William J Buchanan. 2023. Hamming distributions of popular perceptual hashing techniques. *Forensic Science International: Digital Investigation* 44 (2023), 301509.
- [30] S. Morgan. 2024. The World Will Store 200 Zettabytes Of Data By 2025. <https://cybersecurityventures.com/the-world-will-store-200-zettabytes-of-data-by-2025/>.
- [31] J. Ning, X. Huang, G. S. Poh, S. Xu, J. Loh, J. Weng, and R. H. Deng. 2020. Pine: Enabling privacy-preserving deep packet inspection on TLS with rule-hiding and fast connection establishment. In *25th European Symposium on Research in Computer Security (Lecture Notes in Computer Science, Vol. 12308)*.
- [32] J. Ning, G. S. Poh, J. Loh, J. Chia, and E. Chang. 2019. PrivDPI: Privacy-preserving encrypted traffic inspection with reusable obfuscated rules. In *ACM Conference on Computer and Communications Security*.
- [33] U.S. Department of Justice. 2020. International Statement: End-To-End Encryption and Public Safety. <https://www.justice.gov/archives/opa/pr/international-statement-end-end-encryption-and-public-safety>.
- [34] U.S. Department of Justice. 2023. Independence Man Sentenced for Child Pornography. <https://www.justice.gov/usao-wdmo/pr/independence-man-sentenced-child-pornography-1>.
- [35] U.S. Department of Justice. 2023. Man Stored Child Pornography on Google Account, Sentenced to 14 Years in Federal Prison. <https://www.justice.gov/usao-wdtx/pr/man-stored-child-pornography-google-account-sentenced-14-years-federal-prison>.
- [36] E Ong and J Kubiatowicz. 2005. Optimizing robustness while generating shared secret safe primes. In *Public Key Cryptography – PKC*. Springer.
- [37] Maxim Raginsky and Svetlana Lazebnik. 2009. Locality-sensitive binary codes from shift-invariant kernels. In *Advances in Neural Information Processing Systems*.
- [38] Sarah Scheffler, Anunay Kulshrestha, and Jonathan Mayer. 2023. Public verification for private hash matching. In *IEEE Symposium on Security and Privacy*. IEEE, 253–273.
- [39] Sarah Scheffler and Jonathan Mayer. 2023. SoK: Content Moderation for End-to-End Encryption. *Proceedings on Privacy Enhancing Technologies* 2023, 2 (2023), 1501–1508.

- 403–429.
- [40] Sefik Ilkin Serengil and Alper Ozpinar. 2025. LightPHE: Integrating Partially Homomorphic Encryption into Python with Extensive Cloud Environment Evaluations. *arXiv preprint arXiv:2408.05219* (2025). doi: 10.48550/arXiv.2408.05219. [Online]. Available: <https://arxiv.org/abs/2408.05219>.
- [41] Shawn Shan, Jenna Cryan, Emily Wenger, Haitao Zheng, Rana Hanocka, and Ben Y Zhao. 2023. Glaze: Protecting artists from style mimicry by {Text-to-Image} models. In *32nd USENIX Security Symposium (USENIX Security 23)*. 2187–2204.
- [42] J. Sherry, C. Lan, R. Ada Popa, and S. Ratnasamy. 2015. BlindBox: Deep packet inspection over encrypted traffic. In *ACM SIGCOMM*. 213–226.
- [43] Lukas Struppek, Dominik Hintersdorf, Daniel Neider, and Kristian Kersting. 2022. Learning to break deep perceptual hashing: The use case neuralhash. In *ACM Conference on Fairness, Accountability, and Transparency*. 58–69.
- [44] J. Wen, J. Liu, A. Wu, and J. Weng. 2021. A general framework for matching pattern hiding in deep packet inspection. In *22nd International Conference on Information Security Applications (Lecture Notes in Computer Science, Vol. 13009)*.
- [45] R. Williams. 2022. The US now hosts more child sexual abuse material online than any other country. <https://www.technologyreview.com/2022/04/26/1051282/the-us-now-hosts-more-child-sexual-abuse-material-online-than-any-other-country/>.
- [46] Yuchen Yang, Qichang Liu, Christopher Brix, Huan Zhang, and Yinzhi Cao. 2025. {CertPHash}: Towards Certified Perceptual Hashing via Robust Training. In *USENIX Security Symposium*. 7839–7856.

A Open Science

We adhere to the open science policy and our implementation is available (anonymized) at [1].

B Distributed Signatures

Fig. 7 describes the distributed key generation and signing protocols for our set homomorphic signature scheme. Since the protocol simply uses the protocols of Algesheimer et al. [2] to compute a shared modulus, and Catalano et al. [11] to invert values module a shared modulus, we refer to their works for a full security proof.

C Authenticated OPRF

We describe the protocol realizing $\mathcal{F}_{\text{authOPRF}}$ in Fig. 8. The security of the protocol reduces to the secure of the 2HashDH OPRF construction [22] and the unforgeability of the randomizable set homomorphic scheme, proven in Theorem E.1

D Security Definitions

D.1 Weak Random Key Robustness

We will use a symmetric key encryption scheme defined by the triple of algorithms $\Pi_{\text{enc}} = \langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle$. $\text{KEYGEN}(1^\lambda)$ is a randomized key generation algorithm that on input a security parameter outputs a key κ from the key space K ; ENC is a randomized encryption algorithm that on input a key κ and a message m outputs a ciphertext $c \leftarrow \text{ENC}_\kappa(m)$; DEC is a deterministic algorithm that on input a key κ and ciphertext c outputs the plaintext m . Correctness of the encryption scheme implies $\text{DEC}_\kappa(\text{ENC}_\kappa(m)) = m$.

The security property we will need is random key robustness which implies that a ciphertext encrypted under a randomly chosen key, cannot be decrypted under a different, randomly chosen key. It is known that an encryption scheme that satisfies IND-CPA security and ciphertext integrity has weak random key robustness.

D.1.1 IND-CPA security. This property implies that a PPT. adversary cannot distinguish between a ciphertext produced by an encryption scheme $\Pi_{\text{enc}} = \langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle$ and a random string of equal length. This is defined using the following experiment

$$\begin{array}{ll} \text{EXPT}_{\Pi_{\text{enc}}=\langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle}^{\text{ind-cpa\$-0}}(A) & \text{EXPT}_{\Pi_{\text{enc}}=\langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle}^{\text{ind-cpa\$-1}}(A) \\ \kappa \leftarrow \text{KEYGEN}() & \kappa \leftarrow \text{KEYGEN}() \\ \hat{b} \leftarrow A^{0_\kappa}() & \hat{b} \leftarrow A^{0_\$}() \\ \text{return } \hat{b} & \text{return } \hat{b} \end{array}$$

where the maximum is taken over all adversaries A that run in q_H queries to O .

Here, O_κ is an oracle to which the key κ is a fixed input. In each invocation to it, the adversary A provides an input plaintext m and the oracle returns $\text{ENC}_\kappa(m)$. The oracle $O_\$$ returns random strings of length equal to a ciphertext.

The adversary’s advantage is defined as

$$\begin{aligned} \text{Adv}_{\Pi_{\text{enc}}=\langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle}^{\text{ind-cpa\$}}(A) &= \\ \left| \mathbb{P}\left(\text{EXPT}_{\Pi_{\text{enc}}=\langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle}^{\text{ind-cpa\$-1}}(A) = 1\right) - \mathbb{P}\left(\text{EXPT}_{\Pi_{\text{enc}}=\langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle}^{\text{ind-cpa\$-0}}(A) = 1\right) \right| \\ \text{Adv}_{\Pi_{\text{enc}}=\langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle}^{\text{ind-cpa\$}}(t, q_H) &= \max_A \text{Adv}_{\Pi_{\text{enc}}=\langle \text{KEYGEN}, \text{ENC}, \text{DEC} \rangle}^{\text{ind-cpa\$}}(A) \end{aligned}$$

$\mathcal{F}_{dss}$: Distributed Signing Functionality

Parameters: A randomizable homomorphic signature scheme $\Pi_{RH-Sig} = \langle \text{GEN}, \text{SIGN}, \text{VRFY}, \text{RAND}, \text{AUTH} \rangle$. η auditors $A_1, \dots, A_\eta$.

Inputs: The auditors have a common list of item $\mathcal{B} \subseteq \mathcal{U}$ and public-private key pair $\{pk_k^{(r)}, sk_k^{(r)}\}_{k \in [\eta]}$ for signature scheme Π_{RH-Sig}

Functionality:

- Upon receiving $(\text{DSign}, A_k, \mathcal{B}, sk_k)$ from A_k , store $\mathcal{B}, sk_k$. After receiving the DSign message from all auditors, check that all the messages have the same input $\mathcal{B}$. If so, output $\text{SIGN}_{sk_1, \dots, sk_\eta}(\mathcal{B})$, otherwise abort.
- Upon receiving $(\text{DAdd}, A_k, \mathcal{B}, \mathcal{S}, sk_k)$ from A_k , store $\mathcal{B}, \mathcal{S}, sk_k$. After receiving the DAdd from all auditors, check that all the messages have the same inputs $\mathcal{B}$ and $\mathcal{S}$. If so, output $\text{SIGN}_{sk_1, \dots, sk_\eta}(\mathcal{B} \cup \mathcal{S})$, otherwise abort.
- Upon receiving $(\text{DDEL}, A_k, \mathcal{B}, \mathcal{S}, sk_k)$ from A_k , store $\mathcal{B}, \mathcal{S}, sk_k$. After receiving the DDEL from all auditors, check that all the messages have the same inputs $\mathcal{B}$ and $\mathcal{S}$. If so, output $\text{SIGN}_{sk_1, \dots, sk_\eta}(\mathcal{B} \setminus \mathcal{S})$, otherwise abort.

 Π_{dss} : Distributed Signing Protocol Realizing $\mathcal{F}_{dss}$ **Setup (one-time):**

- Auditors generate a shared modulus N using the protocols of Algesheimer et al. [2], Ong and Kubiawicz [36]. Auditor A_k receives additive shares of the Sofie-Germaine primes $\langle p' \rangle_k, \langle q' \rangle_k$. Using the shares, the auditors compute additive shares of N , and additive shares of the order of the quadratic residue subgroup $\phi(QR_N) = p'q'$, using the BGW protocol [5]
- Auditors select a generator g :
 - (1) Sample a random value $g \in [2, N-1]$ using a distributed randomness beacon [12]
 - (2) Verify (locally) that the Jacobi symbol $(\frac{g}{N}) \neq -1$. If so g is a quadratic non-residue. Go to step (1) and repeat.
 - (3) If $(\frac{g}{N}) = 1$, run a BGW instance with $g^{(p')k}$ and $g^{(q')k}$ as inputs and check if $\prod_{k=1}^{\eta} g^{(p')k} \bmod N \neq 1 \wedge \prod_{k=1}^{\eta} g^{(q')k} \bmod N \neq 1$. If the check is successful, return g as the generator. Otherwise, go to step (1) and repeat.

Signing set $\mathcal{B}$:

- (1) Compute $U \leftarrow \prod_{b_i \in \mathcal{B}} H_p(b_i)$
 - (a) Using the protocol of Catalano et al. [11, Fig 3], compute additive shares of $U^{-1} \bmod \phi(QR_N)$. Auditor A_k gets share $\langle x \rangle_k$.
 - (b) Compute $\sigma_{\mathcal{B}} \leftarrow g^{\sum_{k \in [1, \eta]} \langle x_i \rangle_k}$ using a ring network: Auditor A_k waits to receive $\sigma' \leftarrow g^{\sum_{m \in [1, k-1]} \langle x_i \rangle_m}$ from A_{k-1} . Upon receiving σ' , it forwards $\sigma' g^{\langle x_i \rangle_k} = g^{\sum_{m \in [1, k]} \langle x_i \rangle_m}$ to A_{k+1} . Auditor A_k finally obtains and broadcasts $\sigma_{\mathcal{B}} = \text{SIGN}_{sk_1, \dots, sk_\eta}(\mathcal{B})$

Adding set $\mathcal{S}$ and computing signature on $\mathcal{B} \cup \mathcal{S}$

Use the mechanism described above to sign $\mathcal{S}$, $\sigma_{\mathcal{S}} = \text{SIGN}_{sk_1, \dots, sk_\eta}(\mathcal{S})$. Then, compute $\text{SIGN}_{sk_1, \dots, sk_\eta}(\mathcal{B} \cup \mathcal{S}) = \sigma_{\mathcal{B}} \odot_{pk} \sigma_{\mathcal{S}}$

Removing b and computing signature on $\mathcal{B} \setminus \{b\}$

Compute $\text{SIGN}_{sk_1, \dots, sk_\eta}(\mathcal{B} \setminus \{b\}) = \sigma_{\mathcal{B}}^{H_p(b)} \bmod N$

Figure 7: Protocol for distributed signing realizing $\mathcal{F}_{dss}$ $\Pi_{authOPRF}$: Protocol realizing $\mathcal{F}_{authOPRF}$

Parameters: A group $\mathcal{G}$ with order q where the OM-gap Diffie Hellman problem is hard, a distributed randomizable set homomorphic signature $\Pi_{RH-Sig} = \langle \text{GEN}, \text{SIGN}, \text{VRFY}, \text{RAND}, \text{AUTH} \rangle$, and two random oracle $H_1 : \mathcal{U} \rightarrow \mathcal{G}$ and $H_2 : \{0, 1\}^{|\mathcal{U}|+|\mathcal{G}|} \rightarrow \{0, 1\}^\lambda$ and a collision-resistant hash function $h : \{0, 1\}^\lambda \rightarrow \mathcal{R}$

Auditor:

- (1) A_k samples n random values $r_1, \dots, r_n \leftarrow \mathbb{Z}_q$ and shares with the other auditors. A_k receives from each auditor $A_j, k \neq j, r_{j,1}, \dots, r_{j,n}$
- (2) For each $b_i \in \mathcal{B}$, A_k computes $r'_i \leftarrow \prod_{j \in [1, \eta]} r_{j,i}$, and computes the sets $\mathcal{B}' = \{\langle i, H_1(b_i)^{r'_i} \rangle\}_{i \in [n]}$ and $\mathcal{S} \leftarrow \{\langle i, b_i, r_i \rangle\}_{i \in [n]}$
- (3) Auditors invoke $\mathcal{F}_{dss}$ to sign $\sigma_{\mathcal{B}'} \leftarrow \text{SIGN}_{sk}(\mathcal{B}')$ and publish $\mathcal{B}'$ and $\sigma_{\mathcal{B}'}$.
- (4) Auditors invoke $\mathcal{F}_{dss}$ to sign $\sigma_{\mathcal{S}} \leftarrow \text{SIGN}_{sk}(\mathcal{S})$ and provide it to SERVER.

Client:

- (5) After obtaining $\mathcal{B}'$ and $\sigma_{\mathcal{B}'}$, abort if $\text{VRFY}_{pk}(\mathcal{B}', \sigma_{\mathcal{B}'}) \neq 1$.
- (6) For each $\langle i, H_1(b_i)^{r'_i} \rangle$ in $\mathcal{B}'$, compute and add $\langle i, (H_1(b_i)^{r'_i})^{k_{\text{PRF}}} \rangle$ to set $\mathcal{S}'$. Send $\mathcal{S}'$ to SERVER

Server:

- (7) For every $\langle i, (H_1(b_i)^{r'_i})^{k_{\text{PRF}}} \rangle \in \mathcal{S}'$, let $\langle i, b_i, r_i \rangle \in \mathcal{S}$. Compute $((H_1(b_i)^{r'_i})^{k_{\text{PRF}}})^{r_i^{-1}} = H_1(b_i)^{k_{\text{PRF}}}$ and output $\langle b_i, H_2(b_i, H_1(b_i)^{k_{\text{PRF}}}) \rangle$.

Figure 8: Protocol for authenticated OPRF realizing $\mathcal{F}_{authOPRF}$

where the maximum is taken over all adversaries A running in time at most t and making at most q_H queries.

D.1.2 Ciphertext integrity. This property implies that a PPT. adversary cannot produce a new ciphertext after observing encryptions

of polynomially-many ciphertexts under a randomly-chosen key. This is defined using the following experiment

```

EXPTenc=(KEYGEN,ENC,DEC)cci(A)
κ ← KEYGEN()
c' ← A0κ
Let S be the set of all ciphertexts returned by the oracle Oκ
If c' ≠ reject ∧ c' ∉ S
  then return 1
  else return 0

```

Here, O_κ is an oracle to which the key κ is a fixed input. In each invocation to it, the adversary A provides an input plaintext m and the oracle returns ENC_κ(m). The adversary produces c' and wins the experiment if c' is a valid ciphertext and it has not been produced by O_κ in any previous query.

The adversary's advantage is defined as

$$\text{Adv}_{\text{enc}=(\text{KEYGEN},\text{ENC},\text{DEC})}^{\text{cci}}(\mathcal{A}) = \mathbb{P}\left(\text{EXPT}_{\text{enc}=(\text{KEYGEN},\text{ENC},\text{DEC})}^{\text{cci}}(\mathcal{A}) = 1\right)$$

$$\text{Adv}_{\text{enc}=(\text{KEYGEN},\text{ENC},\text{DEC})}^{\text{cci}}(t, q_H) = \max_{\mathcal{A}} \text{Adv}_{\text{enc}=(\text{KEYGEN},\text{ENC},\text{DEC})}^{\text{cci}}(\mathcal{A})$$

where the maximum is taken over all adversaries A running in time at most t and making at most q_H queries.

D.2 Pseudorandom function

Definition D.1 (Pseudorandom function). Let $g : \mathcal{K} \times \mathcal{D} \rightarrow \mathcal{D}$ be a function family, $\mathcal{F}(\mathcal{D})$ be the set of all functions with domain and range $\mathcal{D}$, and A_{prf} be an algorithm that takes an oracle of type $\mathcal{D} \rightarrow \mathcal{D}$ and returns a bit. Let

```

EXPTgprf-0(Aprf)      EXPTgprf-1(Aprf)
h ←$  $\mathcal{F}(\mathcal{D})$              k ←$  $\mathcal{K}$ , h ← gk
b̂ ← Aprfh(·)( )          b̂ ← Aprfh(·)( )
return b̂                 return b̂

```

Then,

$$\text{Adv}_g^{\text{prf}}(\mathcal{A}_{\text{prf}}) = \left| \mathbb{P}\left(\text{EXPT}_g^{\text{prf-1}}(\mathcal{A}_{\text{prf}}) = 1\right) - \mathbb{P}\left(\text{EXPT}_g^{\text{prf-0}}(\mathcal{A}_{\text{prf}}) = 1\right) \right|$$

$$\text{Adv}_g^{\text{prf}}(t, q_H) = \max_{\mathcal{A}_{\text{prf}}} \text{Adv}_g^{\text{prf}}(\mathcal{A}_{\text{prf}})$$

where the maximum is taken over all adversaries A_{prf} that run in time at most t and make at most q queries to h.

E Proofs for Signature Scheme in Sec. ??

E.1 Proof of Unforgeability

THEOREM E.1. *The signature scheme Π_{Sig} is unforgeable under chosen message attacks, that is assuming that strong QR-RSA problem is hard to solve in group QR_N .*

$$\text{Adv}_{\Pi_{\text{Sig}}}^{\text{uf-cma}}(t, q_H) \leq \frac{q_H^2 \log N}{N} + q_H \text{Adv}_N^{\text{QR-RSA}}(t)$$

Here q_H is the number of distinct queries to SIGN_{sk}(·)

PROOF. We will show that if there is an adversary A_{ufcma} that can forge a signature in time t, then there is an adversary A_{QR-RSA} that solves the strong QR-RSA problem with probability Adv_N^{QR-RSA}(t) in time t. The reduction works as follows. Consider that A_{QR-RSA}

simulates A_{ufcma}'s queries to the signing oracle, and the random oracle H_P. Let S₁, ..., S_{n_s} be all the sets that are queried by A_{ufcma} to the signing oracle. Assume that q_H is the total number of queries to the random oracle that includes queries by the signing oracle when responding to A_{ufcma}, and queries directly by A_{ufcma}. Now assume that A_{ufcma} forges a signature on some value y that is not in any of the sets S₁, ..., S_{n_s}. This happens under two conditions.

- (1) There is some y' ∈ $\bigcup_{i \in [i]} S_i$ such that H_P(y') = H_P(y).
- (2) For all y' ∈ S₁, ..., S_{n_s} H_P(y') ≠ H_P(y).

The first case happens with probability at most $\frac{q_H^2 \log N}{N}$ because H_P maps to prime numbers in [1, N] and there are roughly $\frac{N}{\log N}$ prime numbers less than N.

For the second case, consider the following simulation

- (1) A_{QR-RSA} selects q_H random prime numbers less than N, p₁, ..., p_{q_H} and computes $P \leftarrow \prod_{i \in [1, q_H]} p_i$. Then, it computes $z \leftarrow y^P \bmod N$. A_{QR-RSA} publishes z as part of the public key.
- (2) A_{QR-RSA} selects a random index j ∈ [1, q_H] and programs the random oracle H_P such for the i-th oracle query for all i ≠ j, the random oracle returns p_i. For the j-th query, the random oracle returns a random prime number e ∉ p₁, ..., p_{q_H}.
- (3) A_{QR-RSA} answers the signing oracle queries from A_{ufcma} for the sets S₁, ..., S_{n_s}. Specifically, given some set S, A_{QR-RSA} signs S by computing $P' \leftarrow \{p_i : p_i \in p_1, \dots, p_{q_H}, \forall x \in S, H_P(x) \neq$

$p_i\}$ and returning $X \leftarrow y^{P'} = z^{\left(\frac{1}{\prod_{x \in S} H_P(x)}\right)}$

- (4) If A_{ufcma} forges a signature on S' such that there is some x ∈ S' but x ∉ S₁ ∪ ... ∪ S_{n_s} and H'_P(x) = e, then A_{QR-RSA} learns some value v such that v^e = y^P mod N. Since, e is co-prime to P, we can find the values of a, b using the extended Euclid algorithm such that ae + bP = 1. Then, we have (v^by^a)^e = y. A_{QR-RSA} outputs v^by^a as the e-th root of y

To see that this simulation is indistinguishable first note that in the real experiment when A_{ufcma} queries the set S, its view includes g^{1/U_i} where U_i ← $\prod_{x \in S_i} H_P(x)$ and g is a generator of QR_N. In the

simulation, A_{ufcma}'s view includes $z^{\left(\frac{1}{\prod_{x \in S} H_P(x)}\right)}$. Since y is selected randomly from QR_N, the probability that it is not a generator is negligible in λ. Consequently, z ← y^P is also a generator with overwhelming probability, i.e., as long as P is co-prime to p' and q'. From the above, we have that the signature in the simulation is valid and indistinguishable to A_{ufcma}. Finally, A_{QR-RSA} solves the strong QR-RSA problem using the simulation when it programs the random oracle such that H_P(x) = e, this happens with probability at least 1/q_H. Thus, if A_{ufcma} is successful in the forging a signature in the simulation with probability Adv_{Π_{Sig}}^{uf-cma}(t, q_H), then A_{QR-RSA} solves the strong QR-RSA problem also with probability at least Adv_N^{QR-RSA}(t)/q_H. □

E.2 Proof of Indistinguishable Signature under Randomization (IND-SR)

THEOREM E.2. *The signature scheme Π_{Sig} produces indistinguishable signatures under randomization, that is assuming the strong*

QR-RSA problem is hard to solve in QR_N where N is the product of safe primes.

$$\text{Adv}_{\Pi_{\text{Sig}}}^{\text{ind-sr}}(t, q_H, q'_H) \leq \frac{q'_H}{|QR_N|} + q_H \text{Adv}_N^{\text{QR-RSA}}(t)$$

Here q_H is the number of distinct queries to the random oracle H_P and q'_H is the number of distinct queries to the random oracle H .

PROOF. Consider that $A_{\text{indsr}}[3]$ gets a challenge $\langle g^{(\frac{rH_P(x_b)}{U})} \bmod N, H(g^r \bmod N) \rangle$ where $g^{\frac{rH_P(x_b)}{U}} \bmod N$ is a signature on the set $\mathcal{S}$. Since H is a random oracle and $g^r \bmod N$ is uniformly distributed in QR_N [16], $A_{\text{indsr}}[3]$ gets $g^r \bmod N$ from $H(g^r \bmod N)$ only if it is able to compute it from the challenge, or if it has correctly guessed it in q'_H queries. The latter happens with probability at most $q'_H/|QR_N|$.

If $A_{\text{indsr}}[3]$ can distinguish between $g^{(\frac{rH_P(x_0)}{U})} \bmod N$ and $g^{(\frac{rH_P(x_1)}{U})} \bmod N$ then it can also distinguish between $g^{rH_P(x_0)} \bmod N$ and $g^{rH_P(x_1)} \bmod N$ because $A_{\text{indsr}}[3]$ knows U . Let $r = ap'q' + b$. It is known that b is uniformly distributed in $\mathbb{Z}_{p'q'}$ when $r \leftarrow [1, N^2]$ [16]. Thus, $rH_P(x_b) \bmod p'q'$ is uniformly distributed in $\mathbb{Z}_{p'q'}$ and $g^{rH_P(x_0)} \bmod N$ is statistically indistinguishable from $g^{rH_P(x_1)} \bmod N$ since g is a generator of QR_N .

Finally, if $A_{\text{indsr}}[3]$ queries the random oracle for $g^r \bmod N$ without guessing, then it has computed $g^{rH_P(x_b)H_P(x_b)^{-1} \bmod p'q'} \bmod N$. In this case, $A_{\text{QR-RSA}}$ can solve the strong QR-RSA problem with the help of A_{indsr}^* . Consider the following simulation where $A_{\text{QR-RSA}}$ programs the random oracles H, H_P .

- (1) $A_{\text{QR-RSA}}$ selects q_H random prime numbers less than $N, p_1, \dots, p_{q_H}$ and computes $P \leftarrow \prod_{i \in [1, q_H]} p_i$. Then, it computes $z \leftarrow y^P \bmod N$. $A_{\text{QR-RSA}}$ publishes z as part of the public key.
- (2) $A_{\text{QR-RSA}}$ selects a random index $j \in [1, q_H]$ and programs the random oracle H_P such for the i -th oracle query for all $i \neq j$, the random oracle returns p_i . For the j -th query, the random oracle returns a random prime number $e \notin p_1, \dots, p_{q_H}$.
- (3) $A_{\text{QR-RSA}}$ answers the signing oracle queries from $A_{\text{indsr}}[1]$ and $A_{\text{indsr}}[2]$ for the sets $\mathcal{S}_1, \dots, \mathcal{S}_{n_s}$. Specifically, upon receiving $\mathcal{S}$ from $A_{\text{indsr}}[1]$, $A_{\text{QR-RSA}}$ returns $z^{(\frac{1}{\prod_{x \in \mathcal{S}} H_P(x)})} \bmod N$ (see the proof of Theorem E.1 to see how this is computed). If $H_P(x) = e$ for some $x \in \mathcal{S}$, then the simulation aborts.
- (4) Upon receiving x_0, x_1 from $A_{\text{indsr}}[2]$, $A_{\text{QR-RSA}}$ checks that these elements are not in of any of the sets queried by $A_{\text{indsr}}[1]$ and $A_{\text{indsr}}[2]$. Note that the values of $H_P(x_0)$ and $H_P(x_1)$ are in $p_1, \dots, p_{j-1}, e, p_{j+1}, \dots, p_{q_H}$.
- (5) $A_{\text{QR-RSA}}$ computes $P' \leftarrow \{p_i : p_i \in p_1, \dots, p_{q_H}, \forall x \in \mathcal{S}, H_P(x) \neq p_i\}$.
- (6) $A_{\text{QR-RSA}}$ computes $X \leftarrow y^{r'P'} = z^{(\frac{r'}{\prod_{x \in \mathcal{S}} H_P(x)})}$ where $r' \leftarrow [1, N^2]$ and sends the challenge $\langle X, r' \rangle$ to $A_{\text{indsr}}[3]$ where $r'' \leftarrow \{0, 1\}^\lambda$.
- (7) If $A_{\text{indsr}}[3]$ queries H on some value v such that $v^e = z^{r'} = y^{r'P'}$, then $A_{\text{QR-RSA}}$ computes the values of a, b using the extended

Euclid algorithm such that $ae + br'P' = 1$. Note that e is coprime to $r'P'$ with overwhelming probability. Then, we have $(v^b(y)^a)^e = y$. $A_{\text{QR-RSA}}$ outputs $v^b(y)^a$ as the e -th root of y .

To see that the simulation is indistinguishable, observe that $z^{(\frac{1}{\prod_{x \in \mathcal{S}} H_P(x)})} \bmod N$ is a valid and an indistinguishable signature (see the proof of Theorem E.1 for details). Also, as discussed before $z^{(\frac{r'}{\prod_{x \in \mathcal{S}} H_P(x)})} \bmod N$ is a uniformly sampled element in QR_N and is indistinguishable from $z^{(\frac{rx_b}{\prod_{x \in \mathcal{S}} H_P(x)})} \bmod N$ because $z^{(\frac{x_b}{\prod_{x \in \mathcal{S}} H_P(x)})} \bmod N$ is a generator of QR_N with overwhelming probability. Thus, the simulation succeeds and $A_{\text{QR-RSA}}$ wins when it programs either of the challenges x_0 and x_1 as $H_P(x_0) = e$ or $H_P(x_1) = e$, which happens with probability at least $1/q_H$. $\square$

E.3 Proof of robustness to key recovery

THEOREM E.3. The signature scheme $\Pi_{\text{RH-Sig}}$ is robust against key recovery, that is assuming the strong QR-RSA problem is hard to solve in QR_N where N is the product of safe primes.

$$\text{Adv}_{\Pi_{\text{Sig}}}^{\text{rec}}(t, q_H) \leq \frac{q'_H}{|QR_N|} + 2\text{Adv}_{\Pi_{\text{Sig}}}^{\text{ind-sr}}(t, q_H, q'_H)$$

Here q_H is the number of distinct queries to the random oracle H_P and q'_H is the number of distinct queries to the random oracle H .

PROOF. The proof straightforwardly reduces to the IND-SR security of the signature scheme. Since w is the output of a random oracle, $A_{\text{rec}}[2]$ has either randomly guessed g^r or computed it from $\langle \mathcal{S}, y, \sigma_{\mathcal{S}}, \sigma_{\mathcal{S} \setminus \{y\}}, v, pk, \phi_1 \rangle$. The former case happens with probability at most $\frac{q'_H}{|QR_N|}$. In the latter case, if $A_{\text{rec}}[2]$ produces g^r with probability $\text{Adv}_{\Pi_{\text{Sig}}}^{\text{rec}}(t, q_H)$, then this adversary can be used as a subroutine by $A_{\text{indsr}}[3]$ to win $\text{EXPT}_{\Pi_{\text{Sig}}}^{\text{ind-sr}}[b]$ with advantage at least $1/2\text{Adv}_{\Pi_{\text{Sig}}}^{\text{rec}}(t, q_H)$. $\square$

F Proofs of functionality

THEOREM F.1. Assuming that there are protocols securely realizing $\mathcal{F}_{\text{sign}}$ and $\mathcal{F}_{\text{authOPRF}}$, Π_{EM} securely realizes $\mathcal{F}_{\text{Mod}}$

PROOF. We show that there is a polynomial time simulator that can simulate CLIENT 's and SERVER 's views in Π_{EM}

Simulating CLIENT 's view: The client is semi-honest in our model. The simulator implements the random oracles H_P and H and the simulation proceeds as follows:

- (1) During an upload, the simulator gets ID from the message sent by CLIENT to SERVER with the file.
- (2) When CLIENT queries the random oracle H_P on some $y \in \mathcal{U}$, the simulator returns a random value $r \leftarrow \{0, 1\}^\lambda$ as the output of $H_P(y)$ and stores $\langle y, r \rangle$.
- (3) The simulator sends $\langle \text{UPLOADFILE}, \text{ID}, y \rangle$ and obtain the key κ from the $\mathcal{F}_{\text{Mod}}$.
- (4) When the client queries the random oracle H on g^r , then return the key κ as the output of $H(g^r)$.

- (5) If CLIENT's output tape has $\langle \text{ID}, \text{detected} = \text{TRUE} \rangle$, then the simulator simulates the the fine-grained check by sending a random set of indices $\hat{I} \leftarrow_{\$} [1, \tau]$ to CLIENT.
- (6) If CLIENT's output tape has $\langle \text{ID}, \text{detected} = \text{FALSE} \rangle$, then the simulator simulate the fine-grained by sending a set of indices $\hat{I} \leftarrow_{\$} [1, \tau]$ to CLIENT with probability ϵ_{bf} .

The simulation is indistinguishable from the real protocol execution because CLIENT's messages in the simulation include the r, κ when detected = FALSE. Due to the random oracle assumption, these messages are statistically indistinguishable from the messages received in the protocol from the random oracle. When detected = TRUE, CLIENT receives a random set of indices from the simulation. This is indistinguishable from the actual set of indices I received by CLIENT in the real protocol execution. This is because the items in the Bloom filter bf_m sent by CLIENT are the items $b \in \mathcal{B}$ such that $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(b) \equiv k \bmod \zeta)$. Since $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(b))$ is a pseudorandom value, the distribution of b in bf_m is uniformly random. Thus, their corresponding mappings to buckets are also uniformly random.

The simulator simulates false positives by requesting a fine-grained check with probability ϵ_{bf} . A fine-grained check is necessary, in case of a false positive, when there is a collision in the Bloom filter bf_m between $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(b))$ and $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(y))$ for some $b \in \mathcal{B}$. Again, since these are pseudorandom values, the probability of collision does not depend on CLIENT's input y and b . Therefore, the simulation only needs ϵ_{bf} to simulate a false positive without information about b .

Simulating SERVER's view: SERVER is malicious in our model. The simulation proceeds as follows

- **Publishing the blocklist**

- (1) During the blocklist publishing phase, the simulator gets the items in $\mathcal{B}$ from the Publish message sent by the auditors to $\mathcal{F}_{\text{authOPRF}}$, and programs a random oracle $H_P(b) \leftarrow_{\$} \mathcal{P}$ for each $b \in \mathcal{B}$
- (2) For the message to $\mathcal{F}_{\text{dsign}}$, the simulator answers the queries to H_P .

- **Preprocessing**

- (3) During preprocessing, the simulator adds $\langle b_i, z_i \rangle$ to a list L for each $b_i \in \mathcal{B}$ and $z_i \leftarrow_{\$} G$. The simulator sends $\{z_1, \dots, z_n\}$ to SERVER.
- (4) The simulator gets the encrypted Bloom filters from SERVER's messages to CLIENT $\{\text{ENC}_{\kappa_S}(bf_1), \dots, \text{ENC}_{\kappa_S}(bf_\tau)\}$ after the preprocessing phase

- **Upload**

- The simulator gets the set of SERVER's outputs from a polynomial number of uploads that have two types. Type 1: $\langle \text{detected} = \text{TRUE}, \text{ID}, \text{DATA} \rangle$ and Type 2: $\langle \text{detected} = \text{FALSE}, \emptyset \rangle$
 - (5) For every output of type 1 $\langle \text{detected} = \text{TRUE}, \text{ID}, \text{DATA} \rangle$
 - (a) The simulator computes $y \leftarrow \text{embed}(\text{DATA})$ and sends the message $(\text{CHECKFILE}, \text{ID}, y)$ to $\mathcal{F}_{\text{Mod}}$.
 - (b) If $\mathcal{F}_{\text{Mod}}$ aborts, then the simulation aborts. Otherwise, let κ_f be returned by $\mathcal{F}_{\text{Mod}}$.

- (c) The simulator sends z and $\text{ENC}_{\kappa_S}(bf_k)$ to SERVER where this is an entry y, z in L where $z \equiv k \bmod \zeta$
- (d) If SERVER aborts, then the simulator aborts. Otherwise if SERVER sends I , the simulator computes and sends to SERVER $\sigma_{\text{Part}_m}^{r_m H_P(y)}$, where $r_m \leftarrow_{\$} [1, N^2]$, and $R_m \leftarrow_{\$} \{0, 1\}^\lambda$ for $m \in I$. The simulator stores $\langle r_m, R_m \rangle$
- (e) The simulator answers queries to the random oracle H for SERVER. If SERVER sends a query for g^{r_m} for some r_m used in the previous step, then the simulator responds to the query by outputting $\kappa_f \oplus R_m$. Otherwise, the simulator responds with a random element $\{0, 1\}^\lambda$ and stores the output
- (f) The simulator outputs whatever SERVER outputs
- (6) For every output of Type 2 $\langle \text{detected} = \text{FALSE}, \emptyset \rangle$
 - (a) The simulator samples a random value $\hat{z} \leftarrow_{\$} \{0, 1\}^\lambda$ and sends to server $\hat{z}$ and $\text{ENC}_{\kappa_S}(bf_k)$ where $\hat{z} \equiv k \bmod \zeta$
 - (b) If SERVER aborts, then the simulator abort. Otherwise, receive I from the adversary.
 - (c) The simulator samples from $\hat{y} \notin \mathcal{B}$
 - (d) For each $m \in I$, the simulator computes and sends to SERVER $\sigma_{\text{Part}_m}^{r_m H_P(\hat{y})}$, where $r_m \leftarrow_{\$} [1, N^2]$, and $R_m \leftarrow_{\$} \{0, 1\}^\lambda$ for $m \in I$. The simulator stores $\langle r_m, R_m \rangle$
 - (e) The simulator respond to queries to H from SERVER. If there is a query for g^{r_m} for some r_m selected in the previous step, then the simulator aborts. Otherwise, the simulator responds with a random element $\{0, 1\}^\lambda$ and stores the output
 - (f) The simulator outputs whatever SERVER outputs

The simulation is indistinguishable from the real execution. During processing, SERVER's messages includes outputs of $\mathcal{F}_{\text{authOPRF}}\{\text{PRF}_{\kappa_{\text{PRF}}}^C(b_1), \dots, \text{PRF}_{\kappa_{\text{PRF}}}^C(b_n)\}$ while in the simulation, the SERVER's messages includes $\{z_1, \dots, z_n\}$. These are indistinguishable assuming that the outputs of PRF are indistinguishable from random outputs in $\{0, 1\}^\lambda$.

Now, consider the following two cases during uploads
Type 1 output: In this case, SERVER's messages includes $\text{PRF}_{\kappa_{\text{PRF}}}^C(y)$, $\text{ENC}_{\kappa_S}(bf_k)$, $\sigma_{\text{Part}_m}^{r_m H_P(y)}$, $H(g^{r_m}) \oplus \kappa_f$ in the real execution for all $m \in I$. In the ideal world, SERVER's messages includes z , $\text{ENC}_{\kappa_S}(bf_k)$, $\sigma_{\text{Part}_m}^{r_m H_P(y)}$, R_m for all $m \in I$. Assuming that PRF produces outputs indistinguishable from random $\text{PRF}_{\kappa_{\text{PRF}}}^C(y)$ and z are indistinguishable to the adversary. Also, assuming that H is a random oracle $H(g^{r_m}) \oplus \kappa_f$ is indistinguishable from R_m unless the adversary has queried the random oracle on H on r_m before which happens with negligible probability for a fresh randomness r_m used in every query, since g^{r_m} is a random element in QR_N [16].

Type 2 output: Observe that in the type 2 case, there are two possibilities: i) the check terminates with the fast path, or ii) the check proceeds to a fine grained. Since the messages observed by the adversary in the second case subsumes the messages in the first

case, we argue indistinguishability for this case. Later, we show that the distribution of these cases occurring does not break indistinguishability.

When the protocol performs a fine-grained check, SERVER's messages includes $\text{PRF}_{\kappa_{\text{PRF}}}^C(y)$, $\text{ENC}_{\kappa_S}(bf_k)$, $\sigma_{\text{Part}_m}^{r_m H_P(y)}$, $H(g^{r_m}) \oplus \kappa_f$ such that $y \notin \mathcal{B}$. In the simulation, SERVER's messages include $\hat{z}$, $\text{ENC}_{\kappa_S}(bf_k)$, $\sigma_{\text{Part}_m}^{r_m H_P(\hat{y})}$, R_m for all $m \in \mathcal{I}$. Observe that when $y \notin \mathcal{B}$, the adversary does not know $\text{PRF}_{\kappa_{\text{PRF}}}^C(y)$ in the real execution from the preprocessing phase. Thus, assuming that the PRF produces indistinguishable outputs from random $\text{PRF}_{\kappa_{\text{PRF}}}^C(y)$ is indistinguishable from $\hat{z}$. Also, due to H being a random oracle $H(g^{r_m}) \oplus \kappa_f$ is indistinguishable from R_m as discussed above. Finally, if the adversary can distinguish between $\sigma_{\text{Part}_m}^{r_m H_P(y)}$ and $\sigma_{\text{Part}_m}^{r_m H_P(\hat{y})}$ when $y, \hat{y} \notin \mathcal{B}$, then it violates the IND-SR property of the signature scheme, which happens with negligible probability as per Theorem E.2.

The probability that any of the type 2 cases terminates with the fast path depends on the false positives in the Bloom filter, which in turn depends on potential collisions between $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(b))$ and $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(y))$ for some $b \in \text{Bin}_k$. In the simulation, a false positive occurs when there is a collision between $\text{PRP}_{\kappa_{\text{PRP}}}^S(\text{PRF}_{\kappa_{\text{PRF}}}^C(b))$ and $\text{PRP}_{\kappa_{\text{PRP}}}^S(z)$. Since $\text{PRF}_{\kappa_{\text{PRF}}}^C(b)$ and z are indistinguishable due to the PRF, the probability of a false positive in the simulation is distributed indistinguishably to the probability of a false positive in the real execution. $\square$

THEOREM F.2. *Assuming that there are protocols securely realizing $\mathcal{F}_{\text{dsign}}$ and $\mathcal{F}_{\text{authOPRF}}$, Π_{FM} securely realizes $\mathcal{F}_{\text{Mod}}$*

PROOF. We show that there is a polynomial time simulator that can simulate CLIENT's and SERVER's views in Π_{FM}

Simulating CLIENT's view: Simulating CLIENT's view is straightforward in our model since CLIENT is semi-honest and does not receive any messages from the protocol. The simulator simply responds to queries to H_P and H . CLIENT also receives the encrypted ciphertexts corresponding to the $n\gamma(1 + \epsilon_{\text{CHT}})$ entries in the padded cuckoo hash table, as well as the auditor's signatures on the buckets. Due to IND-CPA security, the ciphertexts can be simulated by sending the encryptions of $n\gamma(1 + \epsilon_{\text{CHT}})$ of $\emptyset$ instead.

Simulating SERVER's view: SERVER is malicious in our model and the simulation proceeds and its only input into the protocol is $\mathcal{B}$. The simulator extracts $\mathcal{B}$ from the random oracle queries sent by the auditors in the publish phase.

In the simulation, the simulator gets the set of SERVER's outputs from a polynomial number of uploads that have three types. Type 1: $\langle \text{detected} = \text{TRUE}, \text{ID}, \text{DATA} \rangle$, Type 2: $\langle \text{detected} = \text{FALSE}, \emptyset \rangle$ and Type 3: Type 2: $\langle \text{detected} = \text{FALSE}, \emptyset \rangle$

For every output of Type 1 $\langle \text{ID}, \text{detected} = \text{TRUE}, \text{DATA} \rangle$

- (1) The simulator selects a random key $\kappa_V \leftarrow \mathcal{K}$ and secret shares it $ss_1, \dots, ss_\gamma$
- (2) The simulator computes $\tilde{y} \leftarrow \text{embed}(\tilde{y})$ and $\pi_j(\tilde{y})$ for $j \in [1, \gamma]$.
- (3) The simulator sends $\langle \text{CHECKFILEFUZZY}, \text{ID}, \tilde{y} \rangle$ to $\mathcal{F}_{\text{Mod}}$ and receives κ_f . The simulation aborts if $\mathcal{F}_{\text{Mod}}$ aborts.
- (4) For each $j \in [1, \gamma]$, if the simulator has responded a H_P query for $\langle j, \cdot, \cdot, \pi_j(\tilde{y}) \rangle$ before, then the simulator sets $u_{1,j} \leftarrow$

$u_{1,j}, u_{2,j} \leftarrow u_{2,j}, v_{1,j} \leftarrow v_{1,j}$ and $v_{2,j} \leftarrow v_{2,j}$ as computed in the real protocol

- (5) For each $j \in [1, \gamma]$, if the simulator has not responded a H_P query for $\langle j, \cdot, \cdot, \pi_j(\tilde{y}) \rangle$ before then the simulator sets $u_{1,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,1})$, $u_{2,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,2})$, $v_{1,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,3})$ and $v_{2,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,4})$, where $m_{j,1}, m_{j,2}, m_{j,3}, m_{j,4} \leftarrow \mathcal{P}$
- (6) The simulation follows the steps of the protocol to generate content verification tokens $C_{2,j,k}$ and $C_{1,j,k}$
- (7) The simulation sets the obfuscated secret shares $ss'_{1,j,k} \leftarrow \{0, 1\}^\lambda$ and $ss'_{2,j,k} \leftarrow \{0, 1\}^\lambda$ for all $j \in [1, \gamma]$, $k \in [1, \ell]$
- (8) The simulator secret shares the key κ_f for each $j \in [1, \gamma]$ similar to the protocol: $\{ss_{1,j,1}, \dots, ss_{1,j,\ell}\} \leftarrow \Pi_{\text{dss}}(\kappa_f). \text{SHARE}$ and $\{ss_{2,j,1}, \dots, ss_{2,j,\ell}\} \leftarrow \Pi_{\text{dss}}(\kappa_f). \text{SHARE}$
- (9) Simulator stores two lists L_1 and L_2 of the random coins used in the above steps containing $\langle r_{1,j,k}, ss'_{1,j,k} \rangle$ and $\langle r_{2,j,k}, ss'_{2,j,k} \rangle$ for all $j \in [1, \gamma]$, $k \in [1, \ell]$
- (10) After sending all the information computed to SERVER, the simulator waits for queries to H_P and H .
- (11) If there is a query to H for some $g^{r_{i,j,k}}$ such that $\langle r_{i,j,k}, ss_{i,j,k} \rangle \in L_i$, then the simulator responds with $ss'_{i,j,k} \oplus ss_{i,j,k}$. Otherwise, it responds with a random value $\{0, 1\}^\lambda$ and stores in the random oracle.
- (12) The simulator outputs whatever SERVER outputs

For every output of Type 2 $\langle \text{ID}, \text{detected} = \text{FALSE}, \emptyset \rangle$

- (1) The simulator selects a random key $\kappa_V \leftarrow \mathcal{K}$ and secret shares it with a t_D -out-of- γ secret sharing scheme $ss_1, \dots, ss_\gamma$
- (2) For each $j \in [1, \gamma]$,
 - (a) The simulator sets $u_{1,j} \leftarrow \text{Enc}_{p_{ke}}(ss_j)$ and $u_{2,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,2})$ where $m_{j,2} \leftarrow \mathcal{P}$
 - (b) If $\mathcal{I} \neq \emptyset$, then the simulator sets $v_{1,j} \leftarrow \text{Enc}_{p_{ke}}(t_k)$ where $t_k \in \mathcal{I}$. Otherwise, it sets $v_{1,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,2})$ where $m_{j,2} \leftarrow \mathcal{P}$. The simulator removes t_k from $\mathcal{I}$.
 - (c) The simulator sets $u_{2,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,4})$ where $m_{j,4} \leftarrow \mathcal{P}$
- (3) The simulation selects some $\tilde{y}' \notin \mathcal{B}$ and follows the steps of the protocol to generate the content verification tokens $C_{2,j,k}$ and $C_{1,j,k}$ for $j \in [1, \gamma]$ and $k \in [1, \ell]$.
- (4) The simulation sets the obfuscated secret shares $ss'_{1,j,k} \leftarrow \{0, 1\}^\lambda$ and $ss'_{2,j,k} \leftarrow \{0, 1\}^\lambda$ for all $j \in [1, \gamma]$, $k \in [1, \ell]$
- (5) After sending all the information computed to SERVER, the simulator waits for queries to H_P and H .
- (6) For each query to H , the simulator responds with a random value $\{0, 1\}^\lambda$ unless it has been responded to before. The simulator stores the response.
- (7) The simulator outputs whatever SERVER outputs

For every output of type 3 $\langle \text{ID}, \text{detected} = \text{FALSE}, \emptyset \rangle$

- (1) For each $j \in [1, \gamma]$, the simulator sets $u_{1,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,1})$, $u_{2,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,2})$, $v_{1,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,3})$ and $v_{2,j} \leftarrow \text{Enc}_{p_{ke}}(m_{j,4})$, where $m_{j,1}, m_{j,2}, m_{j,3}, m_{j,4} \leftarrow \mathcal{P}$
- (2) The simulation selects some $\tilde{y}' \notin \mathcal{B}$ and follows the steps of the protocol to generate the content verification tokens $C_{2,j,k}$ and $C_{1,j,k}$ for $j \in [1, \gamma]$ and $k \in [1, \ell]$.
- (3) The simulation sets the obfuscated secret shares $ss'_{1,j,k} \leftarrow \{0, 1\}^\lambda$ and $ss'_{2,j,k} \leftarrow \{0, 1\}^\lambda$ for all $j \in [1, \gamma]$, $k \in [1, \ell]$

- (4) After sending all the information computed to SERVER, the simulator waits for queries to $H_{\mathcal{P}}$ and H .
- (5) For each query to H , the simulator responds with a random value $\{0, 1\}^{\lambda}$ unless it has been responded to before. The simulator stores the response.
- (6) The simulator outputs whatever SERVER outputs

The simulations for all three cases are indistinguishable from the protocol execution due to IND-CPA guarantees of the additively homomorphic encryption scheme, and IND-SR guarantees of the signature scheme.

□